

Abstract

Diese Arbeit untersucht ein interaktives KI-Assistenzsystem im Vibe-Coding-Kontext im Rahmen einer standardisierten Coding-Challenge in einer kontrollierten Brownfield-Sandbox (BuyIn). Ziel ist es, Integrations- und Abgleichsarbeit im untersuchten Setting empirisch über Artefaktspuren und Interaktionsdaten beschreibbar zu machen, ohne Produktivitätsgewinne zu quantifizieren oder eine Rangordnung von Tools oder Modellen abzuleiten. Leitend sind drei Forschungsfragen zum Zusammenhang von Entwicklerkompetenz als Analyseperspektive mit beobachtbaren Umsetzungsständen, zur Rolle von Code- und Ausführungsartefakten für die Ergebnisbeschreibung sowie zu Chat-Interaktionsmustern im Arbeitsprozess.

Methodisch basiert die Studie auf einer standardisierten Coding-Challenge mit $N = 18$ Teilnehmenden, einer Timebox von 60 Minuten und einer standardisierten Entwicklungsumgebung. Die Auswertung nutzt eine artefaktgestützte Pipeline-Logik mit den Statuskategorien *Attempted*, *Implemented_{static}* und *Execution observed* und trianguliert diese mit Code- und Ausführungsartefakten, Chat-Exports und Survey-Merkmalen. Die Ergebnisdarstellung bleibt überwiegend deskriptiv; es werden keine inferenzstatistischen Gruppenvergleiche durchgeführt und keine kausalen Effekte beansprucht. Die Befunde sind auf das untersuchte Setting bezogen und sind nicht als allgemeingültige Integrations- oder Einführungsempfehlung zu verstehen.

Inhaltsverzeichnis

1. Einleitung	8
1.1. Motivation	8
1.2. Problemstellung	9
1.3. Zielsetzung der Arbeit	9
1.4. Aufbau der Arbeit	11
2. Theoretischer Hintergrund und Stand der Forschung	12
2.1. KI-Assistenzsysteme und das Paradigma des Vibe-Coding	12
2.1.1. Technologische Grundlagen: Transformer und In-Context Learning	12
2.1.2. Definition: Vibe-Coding als System-1-Aktivität	13
2.1.3. Theoretische Einordnung in die HCI-Forschung	13
2.2. Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen . .	14
2.2.1. Das Dreyfus-Modell der Expertise	14
2.2.2. Kritik und Erweiterung: ACT-R und Adaptive Expertise	14
2.2.3. Situated Learning und Cognitive Apprenticeship	15
2.3. Kognitionspsychologische Aspekte der Human-AI Interaction	15
2.3.1. Automation Bias und Vertrauen	15
2.3.2. Cognitive Load Theory (CLT)	16
2.3.3. Mixed-Initiative Interaction	16
2.4. Empirische Studien zu KI-Assistenz	16
2.5. Softwarequalität und Produktivität	16
2.5.1. Softwarequalität nach ISO/IEC 25010	17
2.5.2. Produktivität: Das SPACE-Framework	17
2.6. Zusammenfassung und Forschungslücke	17
3. Forschungsdesign und theoriegeleitete Perspektiven	19
3.1. Methodischer Ansatz: Between-Subjects Design	19
3.2. Theoriegeleitete Perspektiven	19
3.2.1. P1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)	19
3.2.2. P2: Das Abgleich-Paradoxon (Low-Code)	20
3.2.3. P3: Skeptizismus und Qualitätsfokus (High-Coder)	20
3.3. Operationalisierung der Kompetenzstufen	21
3.4. Kontrolle von Störvariablen (Confounder)	22
3.5. Die Sandbox-Umgebung als unabhängige Variable: Ecological Validity . .	22
4. Durchführung und Methodik der Datenerhebung	25
4.1. Ablauf der Untersuchung	25

4.2.	Werkzeuge, Konfiguration und Rahmenbedingungen	26
4.3.	Ethik, Datenschutz und Einwilligung der Teilnehmenden	26
4.4.	Operationalisierung der Metriken	27
4.4.1.	Aufgabenfortschritt und Statusausweise (Pipeline-Status)	27
4.4.2.	Code- und Ausführungsartefakte (Surrogatindikatoren)	27
4.4.3.	Chat-Artefakte (Interaktion im Vibe-Coding)	27
4.5.	Güte der Untersuchung (Threats to Validity)	27
4.5.1.	Konstruktvalidität (Construct Validity)	27
4.5.2.	Interne Validität (Internal Validity)	28
4.5.3.	Externe Validität (External Validity)	28
4.5.4.	Reliabilität (Reliability)	28
4.6.	Datenanalyse-Ansatz	29
5.	Technische Realisierung der Sandbox-Umgebung	30
5.1.	Anforderungen an die Sandbox	30
5.2.	Architektur und Tech-Stack: Kognitive Dimensionen	30
5.2.1.	Backend: Node.js & Express (Layered Architecture)	30
5.2.2.	Frontend: React & Vite	31
5.3.	Technische Funktionsweise von Copilot in der Sandbox	31
5.3.1.	Prompt Construction und Context Retrieval	31
5.3.2.	Token Prediction und Ranking	32
5.4.	Design der Aufgaben (Workload Grading)	32
5.5.	Automatisierung der Auswertung	32
5.5.1.	Das Analyse-Skript (<code>run.js</code>)	32
5.5.2.	Grenzen der LLM-gestützten Code-Analyse	33
5.6.	Relevanz für den Unternehmenskontext (BuyIn)	33
5.7.	Containerisierung und Reproduzierbarkeit	33
5.7.1.	Reproduzierbarkeit / Artefaktzugang	33
6.	Ergebnisse	36
6.1.	Datengrundlage und Toolkontext	36
6.1.1.	Studiensetting und Rahmenbedingungen	36
6.1.2.	Datenquellen und Artefakte	36
6.1.3.	Dateninventar und Vollständigkeit	37
6.1.4.	Zuordnung und Zusammenführung der Daten	37
6.1.5.	Reproduzierbarkeit und Auswertungswerkzeuge	38
6.1.6.	Abgrenzung zu Kapitel 7	38
6.2.	Task-Pipeline-Analyse	38
6.2.1.	Methodische Definitionen	38
6.2.2.	Pipeline-Ergebnisse (T1–T5)	39
6.2.3.	Kurzbefunde pro Task (T1–T5)	40
6.3.	Detaillierte Lösungsmuster pro Task (T1–T5)	40
6.4.	Code-Qualität und technische Metriken	44
6.4.1.	ESLint und Typisierung	44

6.4.2.	Ausführungsausgänge und Stabilität	45
6.4.3.	Architektur- und Strukturindikatoren	45
6.4.4.	Zusammenhänge mit Task-Fortschritt	46
6.4.5.	Kurzfazit: Beobachtete Qualitätsmerkmale	46
6.5.	Chat-Interaktionsmuster	46
6.5.1.	Umfang und Intensität der Interaktion	47
6.5.2.	Struktur der Prompts	47
6.5.3.	Debugging-Interaktionen und Iterationsmuster	48
6.5.4.	Beobachtete Antwortmuster in den Modellantworten (Claude Son- net 4.5)	48
6.5.5.	Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)	49
6.6.	Triangulation der Datenquellen	49
6.6.1.	Pipeline-Fortschritt und Implementationsmuster (Strukturindika- toren)	50
6.6.2.	Pipeline-Fortschritt und technische Metriken	51
6.6.3.	Pipeline-Fortschritt und Chat-Interaktionsmetriken	51
6.6.4.	Pipeline-Fortschritt und Survey-Merkmale	52
6.6.5.	Abgrenzung zu Kapitel 7	52
6.7.	Zusammenfassung der Ergebnisse	53
7.	Diskussion	55
7.1.	Zentrale Erkenntnisse dieser Studie	55
7.2.	Ziel und Abgrenzung der Diskussion	55
7.3.	Beantwortung der Forschungsfragen	57
7.4.	Rückbindung an theoriegeleitete Perspektiven	58
7.4.1.	P1: Mid-Code als möglicher Sweet Spot	58
7.4.2.	P2: Abgleich-Paradoxon bei Low-Code	58
7.4.3.	P3: Skepsis und Qualitätsfokus bei High-Coder	59
7.5.	Kompetenzabhängige Interaktionsmuster	59
7.5.1.	No-Code	60
7.5.2.	Low-Code	60
7.5.3.	Mid-Code	60
7.5.4.	High-Coder	61
7.6.	Interpretation der Pipeline-Muster	61
7.6.1.	Attempted ist nicht gleich Implemented _{static}	61
7.6.2.	Implemented _{static} ist nicht gleich Execution observed	62
7.6.3.	Zur Einordnung von Execution observed (0 Vorkommen)	62
7.7.	Einordnung der Chat-Interaktionsmuster	62
7.7.1.	Warum Chat-Intensität kein Effizienzmaß ist	62
7.7.2.	Warum Debugging-Loops normal sind	63
7.7.3.	Explorative und zielgerichtete Interaktion	63
7.8.	Einordnung von Nutzung und Auswertung	63
7.8.1.	Einordnung im Organisationskontext	63
7.8.2.	Einordnung der Auswertung von KI-Coding-Tools	63

7.8.3. Abgrenzung zu Marketing-Narrativen	64
7.9. Limitationen und Ausblick	64
8. Fazit und Schlussfolgerungen	65
8.1. Ziel und Einordnung des Kapitels	65
8.2. Beantwortung der Forschungsfragen	65
8.2.1. Forschungsfrage 1: Kompetenz und Pipeline-Status	65
8.2.2. Forschungsfrage 2: Code- und Ausführungsartefakte	66
8.2.3. Forschungsfrage 3: Chat-Interaktionsmuster	66
8.3. Zentrale Erkenntnisse der Arbeit	66
8.4. Beitrag der Arbeit	67
8.5. Einordnung im BuyIn-Kontext	68
8.6. Abschließendes Fazit	68
A. Anhang	71
A.1. Dateiliste der in der Arbeit behandelten Artefakte	71
A.2. A) Zentrale Auswertungsartefakte	71
A.3. B) Weitere Analysen und Metriken	71
A.4. C) Auswerteskripte (Reproduktionspfad)	72

1. Einleitung

1.1. Motivation

Große Sprachmodelle (Large Language Models, LLMs) werden zunehmend in der Softwareentwicklung eingesetzt und unterstützen Entwurf, Implementierung und Wartung. Im Folgenden wird ein solches *interaktives* Assistenzsystem als *KI-Assistenzsystem* bezeichnet (engl.: *AI Coding Agent*). Der Begriff wird hier im Sinne eines Assistenzsystems ohne eigene Aufgabenplanung verwendet. In vielen Arbeitsabläufen verschiebt sich der Schwerpunkt von der reinen Codeerzeugung hin zur Auswahl und Integration vorgeschlagener Änderungen. Damit gewinnt eine Arbeitsweise an Relevanz, die häufig als *Vibe-Coding* beschrieben wird. Entwicklerinnen und Entwickler delegieren Teilaufgaben an ein KI-Assistenzsystem und übernehmen verstärkt Aufgaben der semantischen Kontrolle, des Reviews sowie der Einbettung in bestehende Strukturen. Diese Form der Mixed-Initiative-Zusammenarbeit betont weniger syntaktische Details als vielmehr das Verständnis von Anforderungen, Nebenbedingungen und Systemkontext.

Im Brownfield-Kontext ist diese Entwicklung besonders bedeutsam. Ein Großteil praktischer Softwareentwicklung findet in bestehenden Codebasen statt, die technische Schulden, heterogene Architekturen, historische Entscheidungen und operative Randbedingungen aufweisen. Bei der Integration von KI-Assistenzsystemen in etablierte Entwicklungsprozesse werden neben Effizienzgewinnen auch Qualitätssicherung, Nachvollziehbarkeit und Risikobegrenzung als relevante Aspekte diskutiert. Daraus ergibt sich ein Spannungsfeld zwischen schneller Iteration und belastbarem Abgleich.

Für die vorliegende Arbeit ist dabei zentral: Potenzielle Effizienzgewinne werden nicht quantifiziert; im Fokus steht vielmehr, wie Integration, Abgleich und Review im Zusammenspiel mit einem KI-Assistenzsystem empirisch beobachtbar werden.

Wissenschaftlich leistet die Arbeit damit einen Beitrag zur empirischen Auswertung von KI-Assistenzsystemen im Brownfield-Kontext, der über reine Produktivitätsbetrachtungen hinausgeht.

Vor diesem Hintergrund ist der Nutzen eines KI-Assistenzsystems in dieser Arbeit nicht als reine Eigenschaft des Tools zu verstehen, sondern als Zusammenspiel aus (i) Interaktion zwischen Mensch und System, (ii) entwicklerseitiger Kompetenz und (iii) Praktiken des Abgleichs und der Integration. Automation Bias kann dazu beitragen, dass Vorschläge überbewertet oder unzureichend kontrolliert werden, während umgekehrt übermäßige Skepsis potenzielle Nutzenaspekte reduziert. Eine wissenschaftlich belastbare Einordnung macht daher eine Betrachtung erforderlich. Diese verknüpft Kompetenz als Analyseperspektive mit beobachtbaren Artefakten und Ergebnissignalen der Arbeit am Code, ohne daraus kausale Effekte oder Tool-Rangordnungen abzuleiten.

1.2. Problemstellung

In der Diskussion um KI-Assistenzsysteme werden häufig Produktivitätsannahmen formuliert, ohne hinreichend zu differenzieren, wie stark die Interaktion durch Kompetenz, Arbeitsstil und Abgleichspraktiken moderiert wird (Peng u. a. 2023). Unter realitätsnahen Bedingungen bestehender Codebasen und unter Zeitdruck bleibt zudem unklar, in welchem Verhältnis wahrgenommener Fortschritt, implementierte Änderungen und als *Execution observed* ausgewiesenes Verhalten zueinander stehen. Damit entsteht eine Forschungslücke: Kontrollierte Untersuchungen kombinieren selten Brownfield-Settings, eine differenzierte Ergebnislogik entlang einer Pipeline und Kompetenz als explizite Analyseperspektive.

Diese Arbeit ist daher keine Produktivitätsmessung (z.B. Output-Geschwindigkeit, Codeumfang oder Zeitersparnis), sondern eine Analyse von Integrations-, Abgleichs- und Reviewarbeit im Vibe-Coding-Kontext.

Sie trifft keine Aussagen über kausale Effekte und leitet keine Rangordnung von Tools oder Modellen ab.

Eine zentrale methodische Herausforderung liegt in der Messbarkeit von Zielerreichung. Chat-Interaktionen und Planäußerungen im Assistenzdialog sind nicht gleichzusetzen mit funktionsfähiger Implementierung. Ebenso ist eine Implementierung im Code nicht gleichzusetzen mit einem *Execution observed*-Ausweis dafür, dass gewünschtes Verhalten tatsächlich erreicht wurde. Diese Differenzen sind im Kontext von KI-generiertem Code besonders relevant, weil die Geschwindigkeit der Generierung und die Plausibilität der Vorschläge die Notwendigkeit eines getrennten Statusausweises nicht aufheben, sondern eher erhöhen können.

Zur Bearbeitung dieser Messprobleme wird eine Diagnoselogik benötigt, die Ergebnisse schrittweise differenziert und durch mehrere Evidenzquellen absichert. In dieser Arbeit wird dafür eine Pipeline-Logik verwendet, die den Bearbeitungsstand einer Aufgabe als *Attempted*, *Implemented_{static}* oder *Execution observed* beschreibt. Für die Trichterdarstellung wird in Kapitel 6 zusätzlich eine abgeleitete Evidenzstufe *Implemented** verwendet. Diese Status sind als Diagnose- und Ergebnislogik zu verstehen, nicht als Qualitätsurteil. Die Einordnung wird nicht aus einer einzelnen Quelle abgeleitet, sondern trianguliert über Code-Artefakte, Ausführungsartefakte, Chat-Logs und Survey-Daten. Ziel ist eine methodisch robuste, deskriptive Einordnung, die subjektive oder dialogbasierte Signale nicht mit *Execution observed*-Ausweisen gleichsetzt und den Interaktionskontext dennoch berücksichtigt.

1.3. Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Auswertung eines KI-Assistenzsystems im Vibe-Coding-Kontext in einer kontrollierten Brownfield-Sandbox. Zur begrifflichen und methodischen Trennung wird zwischen der Assistenzumgebung GitHub Copilot (Tool), dem zugrundeliegenden LLM Claude Sonnet 4.5 und dem KI-Assistenzsystem als funktionaler Einheit unterschieden. Beobachtungen beziehen sich je nach Datenquelle auf unterschiedliche Ebenen. Ein

Tool- oder Modellvergleich ist nicht Gegenstand der Arbeit. Die Wahl von GitHub Copilot erfolgt aus Gründen der ökologischen Validität: Der Integrationskontext (BuyIn) sowie die Zielsetzung der Arbeit orientieren sich an einer Microsoft-nahen Entwicklungsumgebung (insbesondere VS Code und GitHub-Tooling), in der Copilot als praxisnahe Assistenzumgebung naheliegt. Die Datenerhebung erfolgt in einer kontrollierten Coding-Challenge mit $N = 18$ Teilnehmenden, einer Timebox von 60 Minuten und einer standardisierten, containerisierten Arbeitsumgebung. Die Aufgaben umfassen acht Tasks (T1–T8). In der Ergebnisanalyse werden insbesondere frühe Tasks (T1–T5) berücksichtigt, da das Zeitlimit die Bearbeitungsreihenfolge und -tiefe strukturell beeinflusst.

Gegenstand der Arbeit ist dabei ein *interaktives* KI-Assistenzsystem mit kontinuierlicher menschlicher Kontrolle (Human-in-the-Loop); nicht betrachtet werden autonome Agenten mit selbstständiger Aufgabenplanung, Tool-Auswahl oder Entscheidungsautonomie.

Die Arbeit leistet im Kern vier Beiträge:

- (a) Entwicklung und Verwendung einer Brownfield-Sandbox als Messumgebung für kontrollierte, aber realitätsnahe Aufgaben in einer bestehenden Codebasis.
- (b) Operationalisierung von Aufgabenerfolg über die Pipeline-Status: *extitAttempted*, *Implemented_{static}* und *Execution observed* (sowie *Implemented** als abgeleitete Evidenzstufe) als Trichter- und Diagnoselogik.
- (c) Analyse der Ergebnisse im Kontext von Kompetenzgruppen als Analyseperspektiven (No-Code, Low-Code, Mid-Code, High-Coder), ohne diese Einteilung als Wertung von Personen zu interpretieren.
- (d) Methodische Triangulation über Code-Artefakte, Ausführungsartefakte, Chat-Logs und Survey, um die Differenz zwischen intendiertem Vorgehen, implementierten Änderungen und als *Execution observed* ausgewiesenem Verhalten empirisch einordnen zu können.

Forschungsfragen. Aus diesen Zielsetzungen ergeben sich drei Forschungsfragen:

- **RQ1:** Wie hängt die Entwicklerkompetenz als Analyseperspektive mit dem Erreichen der Pipeline-Status *Attempted*, *Implemented_{static}* und *Execution observed* (sowie der abgeleiteten Evidenzstufe *Implemented**) in einer kontrollierten Brownfield-Sandbox zusammen?
- **RQ2:** Wie tragen Code- und Ausführungsartefakte zur Ergebniseinordnung entlang der Pipeline-Status bei, insbesondere zur Trennung von Implementierung und *Execution observed*-Ausweisen?
- **RQ3:** Welche Chat-Interaktionsmuster treten im Vibe-Coding-Kontext auf, und wie sind sie für die Auswertung eines KI-Assistenzsystems im Hinblick auf Abgleich und Integration einzuordnen?

Die Kompetenzgruppen dienen dabei ausschließlich der analytischen Typisierung; daraus wird keine Rangordnung abgeleitet und die Einleitung nimmt keine Ergebnisse vorweg. Die Aussagen dieser Arbeit beziehen sich auf Branches als Analyseeinheiten und auf aggregierte Muster über Gruppen hinweg, nicht auf eine Leistungsbewertung einzelner Personen. Ergänzend werden theoriegeleitete Perspektiven formuliert, um erwartbare Muster im Spannungsfeld zwischen Unterstützungspotenzial, Abgleichsaufwand und Interaktionsdynamik strukturiert zu beschreiben, ohne Ergebnisse vorwegzunehmen.

1.4. Aufbau der Arbeit

Kapitel 2 führt die theoretischen Grundlagen ein und ordnet Vibe-Coding als Arbeitsstil in relevante Perspektiven ein, darunter Kompetenzmodelle, Human-Computer Interaction, Cognitive Load, Automation Bias sowie etablierte Bezugsrahmen aus Qualitäts- und Produktivitätsdiskursen (u. a. ISO/SPACE). Kapitel 3 beschreibt das Forschungsdesign, die Herleitung theoriegeleiteter Perspektiven und die Operationalisierung zentraler Konstrukte, einschließlich der Kompetenzgruppen als Analyseperspektiven und der Pipeline-Status *Attempted*, *Implemented_{static}* und *Execution observed* (sowie *Implemented**). Kapitel 4 erläutert die Durchführung der Studie, die Datenerhebung und die verwendeten Metriken sowie die Diskussion der Gütekriterien durch die Kombination mehrerer Datenquellen.

Kapitel 5 dokumentiert die technische Brownfield-Sandbox, ihre Architektur und die Automatisierungs- und Auswertungsinfrastruktur, die eine standardisierte, reproduzierbare Messung im kontrollierten Setting ermöglicht. Kapitel 6 präsentiert die Ergebnisse in deskriptiver Form und strukturiert sie entlang der Pipeline-Logik und der triangulierten Evidenz aus Artefakten und Interaktionen. Kapitel 7 diskutiert die Befunde, ordnet sie in den theoretischen Kontext ein und fasst Limitationen zusammen, ohne kausale Schlussfolgerungen über die beobachteten Zusammenhänge hinaus zu beanspruchen. Kapitel 8 schließt die Arbeit mit einer synthetischen Beantwortung der Forschungsfragen und einer Einordnung der Beiträge ab.

Abgrenzung und wissenschaftlicher Beitrag. Im Unterschied zu Teilen der bestehenden Forschung steht in dieser Arbeit weder die Optimierung von Prompts noch der Vergleich von Tools oder Modellen im Vordergrund. Ebenso werden keine Produktivitätsmetriken im Sinne von Ausstoß, Geschwindigkeit oder Zeitersparnis als primäre Zielgrößen herangezogen. Stattdessen wird Vibe-Coding im Brownfield-Setting als Interaktions- und Integrationsprozess untersucht, der sich nicht zuverlässig über einzelne Proxy-Metriken abbilden lässt. Der Beitrag liegt in einer auswertenden Perspektive auf Nutzungstypen (Kompetenzgruppen als Analyseperspektiven), einer evidenzbasierten Ergebnislogik über Pipeline-Status sowie der systematischen Triangulation über Code-, Ausführungs-, Chat- und Survey-Artefakte. Damit wird nachvollziehbar, wie sich intendiertes Vorgehen, implementierte Änderungen und als *Execution observed* ausgewiesenes Verhalten unter Zeitdruck zueinander verhalten, ohne daraus eine Leistungsbewertung einzelner Personen oder eine Rangordnung von Assistenzsystemen abzuleiten.

2. Theoretischer Hintergrund und Stand der Forschung

Dieses Kapitel legt das theoretische Fundament für die Untersuchung der Interaktion zwischen menschlicher Kompetenz und KI-Assistenzsystemen. Es erläutert zunächst das Konzept der *KI-Assistenzsysteme* und grenzt den neuartigen Arbeitsmodus des *Vibe-Coding* von klassischer Softwareentwicklung ab (Abschnitt 2.1). Darauf aufbauend wird ein Kompetenzmodell eingeführt, das auf dem Dreyfus-Modell der Expertise basiert, jedoch kritisch um moderne kognitive Architekturen wie ACT-R erweitert wird (Abschnitt 2.2). Anschließend werden kognitionspsychologische Aspekte der Mensch-Maschine-Interaktion – insbesondere *Dual-Process Theory*, *Cognitive Load* und *Automation Bias* – beleuchtet, die erklären, warum unterschiedliche Kompetenzstufen unterschiedlich stark von KI profitieren (Abschnitt 2.3). Abschließend werden Qualitäts- und Produktivitätsmodelle (ISO 25010, SPACE) vorgestellt, die als Begriffs- und Einordnungsrahmen dienen (Abschnitt 2.5).

2.1. KI-Assistenzsysteme und das Paradigma des Vibe-Coding

Die Integration von *Large Language Models (LLMs)* in die Softwareentwicklung markiert einen Paradigmenwechsel. Während klassische IDE-Features wie IntelliSense deterministisch und syntaktisch operieren, arbeiten KI-Assistenzsysteme probabilistisch und semantisch.

2.1.1. Technologische Grundlagen: Transformer und In-Context Learning

Moderne Assistenzumgebungen wie GitHub Copilot basieren auf der Transformer-Architektur (Vaswani u. a. 2017), mit der Abhängigkeiten in Sequenzen über lange Distanzen modelliert werden. Technisch relevant für die Interaktion ist das Fill-In-The-Middle (FIM) Paradigma (Bavarian u. a. 2022). Das Modell sagt nicht nur das nächste Token voraus, sondern berücksichtigt den Kontext *vor* und *nach* der Cursor-Position.

Zusätzlich nutzen diese Systeme Retrieval-Augmented Generation (RAG)-ähnliche Mechanismen. In Assistenzumgebungen wird lokaler Kontext (z.B. geöffnete Tabs, importierte Module) typischerweise als zusätzlicher Prompt-Kontext herangezogen, ohne dass Nutzer dies im Detail steuern. Damit wird die Notwendigkeit für manuelles Prompt

Engineering tendenziell reduziert, zugleich steigt die Abhängigkeit von einer sauberen Projektstruktur. Entsprechend wird in der Literatur die Kontextabhängigkeit von Vorschlägen betont (Witte u. a. 2023).

2.1.2. Definition: Vibe-Coding als System-1-Aktivität

Der Begriff *Vibe-Coding* beschreibt einen emergenten Entwicklungsstil, bei dem der Fokus von der imperativen Syntax-Erstellung zur deklarativen Intentions-Beschreibung verschiebt. Ich definiere Vibe-Coding für diese Arbeit als:

„Einen iterativen Prozess der Softwareerstellung, bei dem der Mensch primär als Architekt und Reviewer agiert, während die KI die Implementierungsdetails auf Basis von natürlichsprachlichen oder kontextuellen ‚Vibes‘ (Intentionen) generiert.“

Dieser Modus lässt sich durch die *Dual-Process Theory* von Kahneman (Kahneman 2011) erklären:

- **System 1 (Schnelles Denken):** Vibe-Coding appelliert an die Intuition. In der Assistenzumgebung werden Vorschläge angezeigt (z.B. nach Eingabe eines Funktionsnamens), die heuristisch akzeptiert werden können („Sieht gut aus“).
- **System 2 (Langsames Denken):** Der Abgleich des Codes mit Anforderungen geht mit analytischer Anstrengung einher. Als Risiko wird diskutiert, dass plausibel wirkende Vorschläge System 2 seltener aktivieren können, was Fehlanpassungen begünstigt.

Dieser Shift verändert die Anforderungen fundamental:

- **Shift from Writing to Reading:** Der Anteil des Code-Lesens und -Verstehens nimmt deutlich zu.
- **Shift from Syntax to Semantics:** Entwickler:innen benötigen weniger Detailwissen darüber, wie eine Schleife syntaktisch korrekt ist; zugleich ist ein präziseres Verständnis dafür erforderlich, was sie logisch leistet.

2.1.3. Theoretische Einordnung in die HCI-Forschung

In der Human-Computer Interaction (HCI) lässt sich Vibe-Coding als eine Form der End-User Development (EUD) verstehen, bei der die Barriere zur Code-Erstellung gesenkt wird. Es weist Parallelen zum Natural Language Programming auf, unterscheidet sich jedoch durch den Mixed-Initiative-Charakter (Horvitz 1999): In der Assistenzumgebung werden proaktive Lösungsvorschläge angezeigt (Ghost Text), während Nutzende fortlaufend auswählen, übernehmen oder verwerfen. Damit wird eine fortlaufende Synchronisation der mentalen Modelle zwischen Mensch und Maschine erforderlich.

2.2. Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen

Um den Einfluss der menschlichen Kompetenz empirisch zu erfassen, ist eine fundierte Klassifizierung zweckmäßig.

2.2.1. Das Dreyfus-Modell der Expertise

Das *Dreyfus-Modell der Expertise* (H. L. Dreyfus und S. E. Dreyfus 1986) beschreibt den Erwerb von Fähigkeiten in fünf Stufen, von der strikten Regelbefolgung bis zur intuitiven Meisterschaft. Für diese Arbeit werden vier Stufen adaptiert:

1. **Novice (No-Code):** Befolgt strikte Regeln, hat kein kontextuelles Verständnis. Verlässt sich vollständig auf die KI als „Black Box“.
2. **Advanced Beginner (Low-Code):** Kann einfache Aufgaben lösen, erkennt aber keine übergeordneten Muster. Kann KI-Vorschläge syntaktisch, aber oft nicht semantisch einordnen.
3. **Competent (Mid-Code):** Arbeitet zielorientiert und kann Probleme in Teilprobleme zerlegen. Nutzt KI als Werkzeug zur Delegation von Syntax- und Boilerplate-Anteilen und besitzt genug Modellverständnis für Abgleich im Kontext.
4. **Proficient/Expert (High-Coder):** Handelt intuitiv und erkennt Abweichungen sofort. Nutzt KI zur Automatisierung von Boilerplate, ist aber skeptisch gegenüber komplexer Logik.

2.2.2. Kritik und Erweiterung: ACT-R und Adaptive Expertise

Das Dreyfus-Modell wird oft als zu linear kritisiert. Es vernachlässigt die *adaptive Expertise* (Hatano & Inagaki), also die Fähigkeit, Wissen auf neue Domänen zu übertragen. Hier bietet die kognitive Architektur **ACT-R** (Adaptive Control of Thought-Rational) von Anderson (1996) eine differenziertere Erklärung:

- **Deklaratives Wissen:** Faktenwissen („Was ist eine Schleife?“). Novizen rufen dieses Wissen häufig mühsam ab.
- **Prozedurales Wissen:** Handlungswissen („Wie schreibe ich eine Schleife?“). Experten haben dies in „Produktionsregeln“ kompiliert, die schnell und mühelos ablaufen.

Relevanz für Copilot: In der Assistenzinteraktion werden *prozedurale Chunks* (fertiger Code) als Vorschläge ausgegeben. Für Novizen, die nur über deklaratives Wissen verfügen, wirkt dies wie eine „Prothese“, die fehlende Produktionsregeln ersetzt. Für Experten ist es eine Beschleunigung bereits vorhandener Regeln.

2.2.3. Situated Learning und Cognitive Apprenticeship

Ergänzend bietet der Ansatz des *Situated Learning* (Lave und Wenger 1991) und *Cognitive Apprenticeship* (Collins u. a. 1989) eine Perspektive auf das Lernen mit KI.

- **Scaffolding:** KI-Unterstützung wird in dieser Perspektive als mögliches „Gerüst“ (Scaffold) diskutiert, das Lernenden das Bearbeiten von Aufgaben nahe ihrer Zone der nächsten Entwicklung erleichtern kann.
- **Legitimate Peripheral Participation:** In dieser Lesart kann KI-Unterstützung frühe Teilhabe an produktiven Prozessen begünstigen, auch wenn die volle Komplexität noch nicht durchdrungen ist.

Die in diesem Theorieblock dargestellten Modelle (u. a. Dreyfus, ACT-R, Adaptive Expertise) dienen in dieser Arbeit als theoretischer Bezugsrahmen zur Einordnung der im empirischen Teil beschriebenen Muster. Eine direkte Operationalisierung einzelner Modellkonstrukte erfolgt nicht; die Modelle werden vielmehr zur begrifflichen Strukturierung und zur theoriegeleiteten Diskussion herangezogen. Die daraus abgeleiteten Einordnungen sind entsprechend an den Umfang und die methodischen Möglichkeiten einer Bachelorarbeit gebunden.

2.3. Kognitionspsychologische Aspekte der Human-AI Interaction

Der Erfolg von KI-Assistenzsystemen hängt maßgeblich von der menschlichen Komponente ab. Theorien der Mensch-Maschine-Interaktion erklären, warum Nutzende unterschiedlich auf KI-Vorschläge reagieren.

2.3.1. Automation Bias und Vertrauen

Ein zentrales Phänomen ist der *Automation Bias* (Parasuraman und Manzey 2010): Die Tendenz, automatisierten Systemen mehr zu vertrauen als der eigenen Urteilkraft. Dies manifestiert sich in zwei Fehlerarten:

- **Errors of Commission:** Der Nutzer akzeptiert einen falschen Vorschlag der KI (häufig bei Low-Code).
- **Errors of Omission:** Der Nutzer übersieht einen Fehler, weil das System keine Warnung ausgibt.

Vertrauen (Trust) wird in der Literatur als kalibriert verstanden (Lee und See 2004). Overtrust kann zu unkritischer Übernahme beitragen, Distrust (oft bei Experten) zu ineffizienter manueller Durchsicht.

2.3.2. Cognitive Load Theory (CLT)

Die Cognitive Load Theory (Sweller 1988) unterscheidet drei Arten der kognitiven Belastung:

- **Intrinsic Load:** Die inhärente Komplexität der Aufgabe (z.B. Rekursion verstehen).
- **Extraneous Load:** Belastung durch die Darstellung oder das Werkzeug (z.B. komplexe IDE-Menüs).
- **Germane Load:** Die kognitive Kapazität, die für das Lernen und Schema-Bildung genutzt wird.

In der Literatur wird eine potenzielle Senkung von Extraneous Load (z.B. Syntax-Details) durch KI-Assistenz diskutiert. Damit wird eine Freisetzung von Germane Load beschrieben, die für den Abgleich genutzt werden kann. Wenn Novizen keine mentalen Modelle (Schemata) haben, können sie diese freigewordene Kapazität nicht nutzen, und der Intrinsic Load des Abgleichs überfordert sie. **Kritik:** Die CLT ist schwer in Echtzeit zu messen. Dennoch bietet sie ein robustes Erklärungsmodell für die Überforderung von Novizen trotz KI-Hilfe.

2.3.3. Mixed-Initiative Interaction

Die Interaktion mit Copilot lässt sich als Beispiel für Mixed-Initiative Interaction (Horvitz 1999) einordnen. In der Assistenzumgebung werden proaktive Code-Vorschläge angezeigt (Ghost Text), die Nutzende annehmen, ablehnen oder durch Weitertippen modifizieren können. Die Effizienz solcher Kollaboration wird in der Literatur u.a. in Bezug auf die Fähigkeit diskutiert, die Intention von Vorschlägen zu antizipieren und zu steuern (Theory of Mind für die KI).

2.4. Empirische Studien zu KI-Assistenz

Aktuelle Studien berichten theoretisch konsistente Befunde, zeigen aber auch neue Phänomene:

- **Grounded Copilot:** Barke u. a. (2023) identifizierten zwei Modi der Interaktion: *Acceleration* (schnelleres Tippen bekannter Logik) und *Exploration* (Nutzen der KI, um neue APIs zu entdecken).
- **Reading Between the Lines:** Mozannar u. a. (2022) zeigten, dass das Modifizieren von KI-Code oft kognitiv teurer ist als das Selbstschreiben, wenn der Vorschlag subtile Fehler enthält.

2.5. Softwarequalität und Produktivität

Die Einordnung der Arbeit mit KI-Assistenzsystemen bedarf klarer Kriterien für Qualität und Produktivität.

2.5.1. Softwarequalität nach ISO/IEC 25010

Der Standard ISO/IEC 25010 definiert Softwarequalität über acht Hauptmerkmale (ISO/IEC 2011). Für die Einordnung von KI-generiertem Code sind insbesondere relevant:

- **Funktionale Eignung:** Erfüllt der generierte Code die fachlichen Anforderungen korrekt?
- **Wartbarkeit:** Ist der Code modular, verständlich und konsistent strukturiert? Studien berichten Tendenzen zur Duplikation statt Abstraktion im Zusammenhang mit KI-gestützter Codeerzeugung, was die Wartbarkeit langfristig gefährden kann (Nadi u. a. 2022).
- **Sicherheit:** Enthält der Code Schwachstellen? In der Literatur wird diskutiert, dass im Kontext öffentlich trainierter LLMs unsichere Muster in Vorschlägen auftreten bzw. repliziert werden können (Pearce u. a. 2022).

2.5.2. Produktivität: Das SPACE-Framework

Produktivität in der Softwareentwicklung ist mehrdimensional. Das SPACE-Framework (Forsgren u. a. 2021) unterscheidet fünf Dimensionen, die auch für den Einsatz von Copilot zentral sind:

- **S (Satisfaction):** Wie werden Zufriedenheit und Frustration im Toolkontext wahrgenommen?
- **P (Performance):** Führt der Einsatz zu qualitativ besseren Ergebnissen?
- **A (Activity):** Wie viele Aufgaben können in einer bestimmten Zeit abgeschlossen werden?
- **C (Communication):** Wie verändert sich die Zusammenarbeit (hier: Mensch-Maschine-Interaktion)?
- **E (Efficiency & Flow):** Wird der Arbeitsfluss durch unterbrechungsfreie Vorschläge gefördert?

2.6. Zusammenfassung und Forschungslücke

Bisherige Studien konzentrierten sich oft auf die technische Leistungsfähigkeit von Modellen oder allgemeine Produktivitätsgewinne (Peng u. a. 2023). Es fehlt jedoch an einer differenzierten Betrachtung, wie die *individuelle Kompetenz* die Interaktion mit dem KI-Assistenzsystem moderiert. In dieser Arbeit wird Kompetenz theoretisch über Dreyfus/ACT-R gerahmt (Abschnitt 2.2) und empirisch über eine transparente Pre-Survey-Heuristik als Analyseperspektive operationalisiert (Kapitel 3). Insbesondere die Frage, ob KI als „Equalizer“ wirkt (Novizen auf Experten-Niveau hebt) oder die Kluft

vergrößert („Rich get richer“), ist ungeklärt. Diese Arbeit adressiert diese Lücke durch eine kontrollierte explorative Untersuchung, die kognitive Theorien (Dual-Process, CLT) mit artefaktgestützten Auswertungen im Software-Engineering-Kontext verbindet.

3. Forschungsdesign und theoriegeleitete Perspektiven

Dieses Kapitel leitet das empirische Design der Studie aus den in Kapitel 2 vorgestellten Theorien ab. Ziel ist es, den Zusammenhang der unabhängigen Variable *Kompetenz* mit den beobachteten Dimensionen *Aufgabenfortschritt/Statusausweise* (artefaktgebunden sowie automatisiert erfasst) sowie *Interaktion* in einer kontrollierten Brownfield-Umgebung zu beschreiben.

3.1. Methodischer Ansatz: Between-Subjects Design

Die Untersuchung folgt einem *Between-Subjects Design*, bei dem Teilnehmende basierend auf ihrer Vorkenntnis in disjunkte Gruppen eingeteilt werden. Begründung nach Wohlin u. a. (2012):

- Vermeidung von Lerneffekten (Carry-Over Effects): Ein Within-Subjects Design (jede Person löst Aufgaben einmal mit, einmal ohne KI) wäre hier problematisch, da das Lösen einer Aufgabe Wissen für den zweiten Durchgang generiert.
- Isolierung der Kompetenzvariable: Das Ziel ist der Vergleich der Interaktionsmuster zwischen verschiedenen Kompetenzniveaus („Mensch A + Maschine“ vs. „Mensch B + Maschine“), nicht der Vergleich des Tools an sich.

3.2. Theoriegeleitete Perspektiven

Die Perspektiven werden direkt aus dem Dreyfus-Modell, der Dual-Process Theory und der Cognitive Load Theory (siehe Abschnitt 2.2 und 2.3) abgeleitet.

Zur Orientierung wird die primäre Zuordnung der Perspektiven zu den Forschungsfragen explizit gemacht: P1 adressiert primär RQ1, P2 adressiert primär RQ1 und RQ2, und P3 adressiert primär RQ3. Diese Zuordnung dient ausschließlich der Strukturierung und nimmt keine Ergebnisse vorweg.

3.2.1. P1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)

Theoretische Begründung: Nach der Cognitive Load Theory profitieren Lernende am meisten, wenn *Extraneous Load* (Syntax) reduziert wird, solange genug Schema-Wissen vorhanden ist, um Output im Kontext einzuordnen. Mid-Code-Teilnehmende befinden

sich in einer Phase, in der sie Konzepte verstehen, aber die syntaktische Umsetzung noch kognitive Ressourcen bindet. In dieser Perspektive wird die Assistenzinteraktion als Delegation von Syntaxarbeit verstanden; dadurch kann *Germane Load* für architektonische Entscheidungen freigesetzt werden. **Alternative Perspektive:** Ein „Rising Tide“-Narrativ würde nahelegen, dass alle Gruppen gleichermaßen profitieren. Dies passt jedoch nur eingeschränkt zur Theorie der *Zone der nächsten Entwicklung*, da Novizen ohne Scaffolding überfordert sein können.

Für die deskriptive Einordnung wird betrachtet, wie häufig hoher Pipeline-Fortschritt innerhalb des 60-Minuten-Limits in den Kompetenzgruppen auftritt (operationalisiert als maximale Anzahl `implemented`-Markierungen pro Branch in `evaluation/task_pipeline_v3.json`; `implemented` wird dabei als heuristisches Artefakt-Signal *Implemented_{static}* verstanden).

3.2.2. P2: Das Abgleich-Paradoxon (Low-Code)

Theoretische Begründung: Novizen und Advanced Beginners (Dreyfus Stufe 1–2) fehlt das mentale Modell, um subtile Fehler zu erkennen. Sie operieren primär im *System 1* (intuitiv) und akzeptieren plausibel klingende Vorschläge unkritisch (*Errors of Commission*). Da ihnen die Schemata fehlen, um den Code zu „parsen“, ist der *Intrinsic Load* des Abgleichs hoch. **Empirische Bezugnahme:** Studien von Vaithilingam u. a. (2022) berichten, dass Nutzer oft Code akzeptieren, den sie nicht verstehen, was zu schwer findbaren Bugs führt.

Für die deskriptive Einordnung wird betrachtet, wie sich Pipeline-Fortschritt zwischen den Kompetenzgruppen verteilt (operationalisiert über die Anzahl `implemented`-Markierungen pro Branch in `evaluation/task_pipeline_v3.json`; *Implemented_{static}* als heuristisches Artefakt-Signal). Aussagen zu Sicherheitslücken, Wartbarkeit oder *Overtrust* werden in dieser Studie als *explorativ* behandelt.

3.2.3. P3: Skeptizismus und Qualitätsfokus (High-Coder)

Theoretische Begründung: Experten (Dreyfus Stufe 4–5) haben starke mentale Modelle und vertrauen der Automation weniger. Sie aktivieren *System 2* (analytisch) häufiger, um Vorschläge zu kontrollieren. Dies kann mit einem *Distrust*-Verhalten einhergehen, bei dem potenzielle Zeitersparnisse durch manuelle Reviews teilweise relativiert werden. **Alternative Sichtweise:** Man könnte annehmen, Experten seien am schnellsten. Die Theorie der *Expertise Reversal Effect* (Kalyuga et al.) legt jedoch nahe, dass Hilfestellungen für Experten redundant und störend sein können.

Für die deskriptive Einordnung wird betrachtet, wie häufig maximaler Pipeline-Fortschritt in den Kompetenzgruppen auftritt (operationalisiert über die Anzahl als `implemented` markierter Tasks pro Branch in `evaluation/task_pipeline_v3.json`; *Implemented_{static}* als heuristisches Artefakt-Signal).

Aussagen zu selektiver Nutzung (z.B. „Accept All“), Codier-Geschwindigkeit und Codequalität (Wartbarkeit/Sicherheit) werden in dieser Studie als *explorativ* behandelt.

Die Perspektiven werden in dieser Arbeit theoriegeleitet formuliert und im Sinne einer explorativen Strukturierung der Ergebnisdiskussion verwendet. Eine inferenzstatistische Auswertung erfolgt nicht; die Perspektiven dienen der geordneten Einordnung der in Kapitel 6 berichteten Befundlinien im Kontext der Forschungsfragen. Entsprechend werden die Perspektiven als deskriptive, theoriegeleitete Deutungsfolie unter den dokumentierten Studienbedingungen herangezogen, ohne kausale Effekte oder gruppenstabile Leistungsrangfolgen zu beanspruchen.

3.3. Operationalisierung der Kompetenzstufen

Die Kompetenzgruppen (No-Code/Low-Code/Mid-Code/High-Coder) werden in dieser Arbeit ausschließlich als *Analyseperspektiven* verwendet (Stratifizierung/Segmentierung) und nicht als Leistungsbewertung einzelner Personen. Die Gruppenzuordnung erfolgt auf Basis eines transparenten, auditierbaren Scoring-Modells mit insgesamt 0–13 Punkten. Die Items werden im Pre-Survey erhoben und in Punkte gemappt; die Summe definiert die Kompetenzstufe. Formale Bildungsindikatoren (z.B. Ausbildungsstand/Abschluss) werden bewusst nicht erhoben und nicht verwendet.

Hinweis: Das Scoring stellt eine pragmatische Heuristik dar und ist kein psychometrisch fundiertes Instrument.

Schwellenwerte zur Gruppenzuordnung. Aus dem Gesamtscore $S \in [0, 13]$ ergeben sich die Kompetenzgruppen wie folgt: **No-Code** ($S = 0$), **Low-Code** ($S = 1$ bis 6), **Mid-Code** ($S = 7$ bis 10) und **High-Coder** ($S = 11$ bis 13). Diese Cutoffs werden *a priori* festgelegt, um nachträgliche Anpassungen an die beobachteten Ergebnisse zu vermeiden.

Reduktion von Self-Report-Bias und Plausibilitätschecks. Da die Operationalisierung auf Selbstauskunft beruht, werden ausschließlich diskrete Erfahrungsangaben (Jahre allgemeine Programmiererfahrung; Jahre TS/Node/Backend) und zwei Likert-Proxies (Implementierungssicherheit; Debugging-/Testpraxis) in ein festes Punkteschema *ohne* nachträgliche Einzelfallkorrekturen übersetzt.

Optionale Sensitivitätsanalyse (Future Work). Als optionale Erweiterung (nicht Teil dieser Arbeit) könnte in zukünftiger Arbeit untersucht werden, wie stabil die beschriebenen Muster gegen plausible Alternativen der Kompetenz-Operationalisierung sind (z. B. leicht verschobene Cutoffs oder Variationen der Punktmappings der Selbsteinschätzungsitems). Solche Sensitivitätsläufe würden hier lediglich die Abhängigkeit der Interpretation von der gewählten Gruppendifinition transparent machen; es wird dabei keine bestimmte Richtung oder Stärke eines Effekts erwartet.

Threats to validity (Kompetenzmessung). Kompetenz ist ein latentes Konstrukt und kann hier nur *approximiert* werden (Konstruktvalidität) – über Selbstauskunft zu allgemeiner Programmiererfahrung, TS/Node/Backend-Erfahrung sowie zwei erfahrungsnahen Selbsteinschätzungs-Proxies (Implementierungssicherheit; Debugging-/Testpraxis).

Selbstauskunft kann systematisch verzerrt sein (Selbstüberschätzung/soziale Erwünschtheit), und Likert-Skalen können zwischen Personen unterschiedlich interpretiert werden (Messinvarianz). Gegenmaßnahmen sind die a priori festgelegte, auditierbare Rubrik mit diskreten Mappings sowie die als optionale Erweiterung vorgeschlagene Sensitivitätsanalyse gegen alternative Cutoffs/Mappings. Die Kompetenzgruppen bleiben damit eine interpretierbare Analyseperspektive, ohne Einzelfallbewertungen abzuleiten.

3.4. Kontrolle von Störvariablen (Confounder)

Um die interne Validität zu sichern, wurden folgende Störvariablen kontrolliert:

- **IDE-Familiarity:** Alle Teilnehmenden nutzten VS Code. Unterschiede in der Vertrautheit wurden im Pre-Survey erfasst.
- **Domain Knowledge:** Das Todo-App-Szenario ist generisch genug, um kein spezifisches Domänenwissen (wie z.B. in der Medizin) vorauszusetzen.
- **Englischkenntnisse:** Da Copilot primär auf englischem Code trainiert ist, wurden Grundkenntnisse vorausgesetzt.

3.5. Die Sandbox-Umgebung als unabhängige Variable: Ecological Validity

Die Wahl einer **Brownfield-Umgebung** (bestehender Code) statt Greenfield (leeres Blatt) ist essenziell für die *ökologische Validität* der Studie. Im industriellen Anwendungskontext arbeiten Entwickler:innen zu ca. 90% in bestehenden Systemen (Brandtner u. a. 2024).

- **Kontext-Sensitivität (RAG-Szenario):** Im Brownfield-Setting sind bestehende Stil- und Strukturkonventionen (z. B. SCSS-Module, React Hooks) relevanter Kontext. Damit werden sowohl Kontexthinweise im Projekt als auch die Praxis adressiert, relevante Dateien zu öffnen und Kontext bereitzustellen.
- **Kognitive Belastung:** Die Teilnehmenden sind mit der Notwendigkeit konfrontiert, sich in der Struktur zurechtzufinden, was für Low-Code-Teilnehmende eine zusätzliche Hürde darstellt (*Intrinsic Load*). Greenfield-Aufgaben (z.B. LeetCode) würden diese Komplexität ausblenden und die Ergebnisse verzerren.

Damit werden *reale* Arbeitsbedingungen simuliert, ohne die Aufgabenbearbeitung auf das Erproben isolierter Algorithmen zu reduzieren.

Input-Quelle	Skala (Pre-Survey)	Punkte (Mapping)	Max. Punkte	Begründung
Jahre allgemeine Programmiererfahrung	Keine Erfahrung Weniger als 1 Jahr 1 bis 3 Jahre Mehr als 3 Jahre	(1)=0 (2)=2 (3)=4 (4)=5	5/13	Proxy für routinierte Programmierpraxis; als Selbstauskunft erhoben und daher kein objektives Kompetenzmaß.
Jahre Erfahrung mit TS/Node/Backend	Keine Erfahrung Weniger als 1 Jahr 1 bis 3 Jahre Mehr als 3 Jahre	(1)=0 (2)=1 (3)=3 (4)=4	4/13	Proxy für Domänen- und Stack-Nähe (relevant für die Brownfield-Sandbox); Selbstauskunft, nicht Leistungsnachweis.
Implementierungssicherheit (Selbsteinschätzung)	Likert 1–5	1–2→0 3→1 4–5→2	2/13	Proxy für wahrgenommene Fähigkeit, Feature-Logik selbstständig umzusetzen; subjektiv und kontextabhängig.
Debugging- und Testpraxis (Selbsteinschätzung)	Likert 1–5	1–2→0 3→1 4–5→2	2/13	Proxy für Validierungs- und Fehlersuchroutinen (z.B. Tests/Debugging) im Arbeitsprozess; Selbstauskunft statt objektiver Messung.

Tabelle 3.1.: Auditierbare Scoring-Rubrik zur Kompetenz-Operationalisierung (0–13 Punkte) basierend auf den tatsächlich erhobenen Pre-Survey-Items (Form Responses 1 in `thesis/surveys/Probanden-Einstufungsformular(Responses).xlsx`). Verwendet werden ausschließlich erfahrungsnahe Proxies (Jahre allgemeine Programmiererfahrung; Jahre TS/Node/Backend; Selbsteinschätzung Implementierungssicherheit; Selbsteinschätzung Debugging-/Testpraxis). Abk.: TS = TypeScript; Node = Node.js; Backend = Serverteil. Die Gewichtung ist als maximal erreichbare Punktzahl pro Item angegeben; der Gesamtscore ist die Summe S .

Gruppe	Dreyfus-Level	Score	Charakteristik
No-Code	Novice	0	Kein mentales Modell von Programmierung. Stützt sich stark auf KI als Black Box.
Low-Code	Advanced Beginner	1–6	Kennt Grundkonzepte, aber keine Architekturmuster. Erhöhtes Risiko für <i>Spaghetti-Code</i> und <i>Automation Bias</i> .
Mid-Code	Competent	7–10	Versteht Zusammenhänge, plant Schritte. Fehler in Vorschlägen sind in dieser Perspektive häufig erkennbar (<i>System 2</i> Aktivierung möglich).
High-Coder	Proficient/Expert	11–13	Intuitives Verständnis. Nutzt KI zur Delegation von Boilerplate, nicht zur Lösungsfindung.

Tabelle 3.2.: Mapping von Kompetenzgruppen auf das Dreyfus-Modell (konzeptionelle, theoriegeleitete Einordnung; idealtypische Kurzcharakteristiken ohne individuelle Leistungszuschreibung).

4. Durchführung und Methodik der Datenerhebung

Während das vorangegangene Kapitel das Forschungsdesign und theoriegeleitete Perspektiven definiert hat, beschreibt dieses Kapitel den konkreten Ablauf der Studie sowie die Operationalisierung der Messgrößen. Es wird detailliert dargelegt, wie die Datenerhebung organisiert wurde, um Aspekte der internen Validität der Ergebnisse nachvollziehbar zu adressieren.

Die in dieser Arbeit verwendete Bezeichnung eines „kontrollierten“ Vorgehens bezieht sich auf die Standardisierung des Ablaufs und der Rahmenbedingungen; sie wird nicht im Sinne eines kausalen Nachweises verwendet. Eine Hypothesenprüfung oder inferenzstatistische Auswertung erfolgt nicht.

4.1. Ablauf der Untersuchung

Die Untersuchung wurde in einem standardisierten Laborsetting durchgeführt. Jede Sitzung folgte einem strikten Protokoll, um Störfaktoren zu minimieren. Der Ablauf gliederte sich in fünf Phasen:

1. **Onboarding (10 Min.):** Die Teilnehmenden erhielten eine Einführung in das Szenario („Sie sind neuer Entwickler im Team...“) und die Entwicklungsumgebung. Es wurde sichergestellt, dass GitHub Copilot aktiviert und funktionsfähig war.
2. **Pre-Survey (5 Min.):** Erfassung der demografischen Daten und Selbsteinschätzung zur Kompetenzeinstufung (siehe Kapitel 3).
3. **Vibe-Coding-Challenge (60 Min.):** Die Kernphase der Untersuchung. Die Teilnehmenden bearbeiteten die Aufgaben selbstständig. Eingriffe durch den Versuchsleiter erfolgten nur bei technischen Problemen (z. B. Absturz der IDE), nicht jedoch bei inhaltlichen Fragen. **Begründung des Zeitlimits:** Die Begrenzung auf 60 Minuten dient der organisatorischen Machbarkeit und fungiert als Stressor. Zeitdruck erhöht die Wahrscheinlichkeit, dass Teilnehmende auf heuristische Strategien (*System 1*) zurückgreifen. Dies kann den *Automation Bias* verstärken und simuliert realistische Deadline-Situationen.
4. **Datensicherung (5 Min.):** Sicherung der Arbeitsartefakte (Branch-Stand, CI-/Lint-Ausgaben und Chat-Export) als Grundlage der späteren Artefaktanalyse.

5. **Debriefing (5 Min.):** Kurzes Abschlussgespräch, um offene Fragen zu klären und qualitatives Feedback einzuholen, das im Survey nicht abgebildet werden konnte.

4.2. Werkzeuge, Konfiguration und Rahmenbedingungen

Die Aufgabenbearbeitung erfolgte in Visual Studio Code als Entwicklungsumgebung. Als KI-Unterstützung wurde GitHub Copilot verwendet; als zugrunde liegendes Modell wurde Claude Sonnet 4.5 genutzt.

Der verwendete Copilot-Plan wird in dieser Arbeit nicht als Untersuchungsvariable geführt, da in der Durchführung eine identische Agentenfunktionalität angesetzt wurde.

Die Bearbeitung erfolgte auf Basis des Repository-Zustands im Main-Branch. Für jede teilnehmende Person wurde ein individueller Branch angelegt und für die Aufgabenbearbeitung genutzt. Vor Beginn der Aufgaben wurden am Ausgangsstand keine inhaltlichen Änderungen vorgenommen; die Sitzung startete aus dem unveränderten Ausgangszustand.

Als Arbeitsgeräte wurden unternehmensinterne BuyIn HP Laptops eingesetzt.

Modellparameter (z.B. „Temperatur“) waren im verwendeten Setup nicht konfigurierbar und wurden entsprechend nicht als konfigurierter Faktor dokumentiert.

4.3. Ethik, Datenschutz und Einwilligung der Teilnehmenden

Die Teilnahme an der Untersuchung erfolgte freiwillig und nach Einwilligung der Teilnehmenden. Der Untersuchungsablauf war so organisiert, dass die Teilnehmenden die Sitzung jederzeit abbrechen konnten.

Im Rahmen der Datenerhebung wurden die im Ablauf beschriebenen Arbeitsartefakte technisch gespeichert, insbesondere der Stand der jeweiligen Arbeits-Branche sowie protokollierte CI-Test- und Lint-Ausgaben. Zusätzlich wurden die Chat-Exporte mit dem KI-Assistenzsystem sowie die im Pre-Survey erhobenen Angaben als Erhebungsdaten abgelegt.

In der schriftlichen Ausarbeitung und in der Analyse werden diese Daten ausschließlich anonym behandelt. Es werden keine Klarnamen oder direkt personenbezogenen Daten (z.B. Kontaktdaten) ausgewertet oder berichtet. Personenbezüge werden in den Ergebnisdarstellungen nicht hergestellt; Auswertungen werden in aggregierter Form (z.B. nach Kompetenzgruppen) oder anhand von nicht-identifizierenden Artefaktmerkmalen beschrieben.

Die Aufgaben und die verwendeten Prompts basierten auf der bereitgestellten Sandbox-Umgebung und den dazugehörigen Aufgabenstellungen. Dabei wurden keine vertraulichen Unternehmensdaten als Inhalte der Aufgaben oder als Eingaben in das KI-System verwendet.

Der Zugriff auf Rohdaten (insbesondere Chat-Exporte und Survey-Exporte) war auf den Autor beschränkt.

4.4. Operationalisierung der Metriken

Zur Einordnung der theoriegeleiteten Perspektiven wurden die abstrakten Beobachtungsdimensionen (Aufgabenfortschritt/Statusausweise sowie Interaktion) in nachvollziehbare, artefaktgebundene Indikatoren übersetzt.

4.4.1. Aufgabenfortschritt und Statusausweise (Pipeline-Status)

Die Ergebnislogik der Arbeit basiert auf einer artefaktgestützten Pipeline-Klassifikation pro Task und Branch. Zur Trennung von Intention, Implementationsartefakt und dem Statusausweis Execution observed werden die Statuskategorien Attempted, Implemented_{static} und Execution observed verwendet (Definitionen und Trichter-Evidenzstufe Implemented* in Abschnitt 6.2). Wichtig ist dabei: Execution observed ist ein methodengebundenes Diagnose-Signal (aus den protokollierten Ausführungsausgaben) und kein allgemeines Qualitätsurteil.

4.4.2. Code- und Ausführungsartefakte (Surrogatindikatoren)

Zur Einordnung der Umsetzungsstände werden zusätzlich struktur- und ausführungsnaher Artefakte herangezogen. Dazu zählen u.a. heuristische Strukturindikatoren und aggregierte Ausführungsausgänge aus den Auswertungsartefakten (z. B. `evaluation/deep_code_analysis.json`, `evaluation/code_quality_metrics.json`). Diese Größen werden durchgängig als Surrogate berichtet und nicht als vollständige Qualitätsmessung verstanden. Terminologische Klarstellung (Evaluation). Der Begriff „Evaluation“ wird in dieser Arbeit ausschließlich im deskriptiven Sinn einer dokumentierten Auswertung/Bestandsaufnahme der Artefakte verwendet und nicht im Sinne einer normativen „Bewertung“.

4.4.3. Chat-Artefakte (Interaktion im Vibe-Coding)

Die Interaktion wird über Chat-Exports und daraus abgeleitete deskriptive Indikatoren beschrieben (z. B. Wortanzahl, Prompt-Antwort-Zyklen, Fehlermarker-Iterationen). Die Auswertung bleibt auf Musterbeschreibung und Triangulation mit Pipeline- und Code-/Ausführungsartefakten beschränkt; Chat-Intensität wird nicht als Effizienz- oder Qualitätsmaß interpretiert.

4.5. Güte der Untersuchung (Threats to Validity)

Die Validität der Studie wird anhand der Kategorien von Cook und Campbell (1979) diskutiert.

4.5.1. Konstruktvalidität (Construct Validity)

Messen wir wirklich das, was wir messen wollen?

- **Problem:** „Kompetenz“ ist schwer zu quantifizieren.
- **Mitigation:** Kompetenz wird als Analyseperspektive über eine auditierbare, diskrete Pre-Survey-Heuristik operationalisiert (Tabelle 3.1). Es wird explizit keine psychometrische Fundierung beansprucht; die Einordnung wird über Triangulation mit Artefaktspuren (Pipeline/Code/Ausführung/Chat) kontextualisiert. Da die Kompetenzzuordnung auf Selbstauskunft im Pre-Survey beruht, sind systematische Verzerrungen (z. B. Selbstüberschätzung/soziale Erwünschtheit und unterschiedliche Interpretation von Skalen) möglich, sodass die Kompetenzgruppen als Analyseperspektiven und nicht als objektives Kompetenzmaß zu verstehen sind.

4.5.2. Interne Validität (Internal Validity)

Können die beobachteten Unterschiede eindeutig der unabhängigen Variable (Kompetenz) zugeschrieben werden?

- **Selection Bias:** Die Zuteilung zu den Gruppen basierte auf Selbsteinschätzung. Um dies zu mitigieren, wurde der Algorithmus zur Klassifizierung strikt angewendet.
- **Experimenter Bias (Rosenthal-Effekt):** Die Erwartungshaltung des Versuchsleiters könnte die Teilnehmenden beeinflussen. **Gegenmaßnahme:** Standardisierte Instruktionen und minimale Interaktion während der Coding-Phase.
- **Compensatory Rivalry (John Henry Effect):** Wenn eine Gruppe sich benachteiligt fühlt, könnte sie sich mehr anstrengen. Da alle Gruppen Copilot nutzten, ist dieses Risiko gering. Dennoch könnten Teilnehmende versuchen, besonders „sorgfältig“ oder „kontrolliert“ zu arbeiten (z. B. durch mehr manuelle Kontrolle), was den beobachteten Fortschritt unter Zeitlimit beeinflussen kann (Saretsky 1972).

4.5.3. Externe Validität (External Validity)

Sind die Ergebnisse verallgemeinerbar?

- **Hawthorne-Effekt:** Das Wissen, beobachtet zu werden, verändert das Verhalten. Dies ist in Laborstudien typischerweise nicht vollständig vermeidbar, wurde aber durch die realistische Aufgabenstellung (Flow-Erleben) abgemildert.
- **Ecological Validity:** Die Nutzung einer Brownfield-Umgebung erhöht die Übertragbarkeit auf Organisationskontexte im Vergleich zu synthetischen LeetCode-Aufgaben deutlich.

4.5.4. Reliabilität (Reliability)

Sind die Ergebnisse reproduzierbar?

- **Reproduzierbarkeit der Auswertung:**

Die Pipeline-Status und Artefakt-Surrogate werden aus versionierten Branches und gespeicherten Auswertungsartefakten abgeleitet. Die Klassifikation folgt festen, dokumentierten Regeln (Attempted, Implemented_{static} und Execution observed; Abschnitt 6.2) und ist damit prinzipiell wiederholbar.

4.6. Datenanalyse-Ansatz

Die Aufgabenserie umfasste die Tasks T1–T8. Unter dem Zeitlimit von 60 Minuten erreichte keine teilnehmende Person Aufgaben über T5 hinaus. Die Tasks wurden so konzipiert, dass sie innerhalb einer Stunde nicht vollständig lösbar sind. Für die Auswertung wurden entsprechend ausschließlich T1–T5 berücksichtigt.

Die Teilnehmenden bearbeiteten die Aufgaben selbstständig, indem sie mithilfe des KI-Agents im jeweils eigenen Branch implementierten. Es wurden keine inhaltlichen Vorgaben, Musterlösungen oder zusätzlichen Einschränkungen zur konkreten Implementationsstrategie gemacht. Die Analyse stützt sich auf die beobachteten Ergebnisse und die gespeicherten Artefakte (z.B. Branch-Status, Ausführungs- und Lint-Ausgaben sowie Chat-Exporte). Die Auswertung erfolgt durch den Autor und wird als explorativ-deskriptiv verstanden; interpretative Einordnungen basieren auf den dokumentierten Artefaktspuren und sind als potenziell subjektiv geprägte Perspektive eine Limitation.

Die Auswertung folgt einem artefaktbasierten, überwiegend deskriptiven Mixed-Methods-Ansatz:

1. **Deskriptive Quantifizierung:** Pipeline-Status (Attempted, Implemented_{static} und Execution observed; sowie Implemented* für die Trichterdarstellung) werden pro Task und Branch berichtet. Ergänzend werden aggregierte Surrogatindikatoren aus Code- und Ausführungsartefakten ausgewiesen.
2. **Musteranalyse & Triangulation:** Chat- und Strukturmuster werden als qualitative Befundlinien beschrieben und mit Pipeline- und Artefaktspuren trianguliert, um Integrations- und Abgleichengpässe im Brownfield-Kontext einzuordnen.

Dieser methodische Aufbau stellt sicher, dass nicht nur das Ergebnis (Code), sondern auch der Prozess (Interaktion) beleuchtet wird.

5. Technische Realisierung der Sandbox-Umgebung

Dieses Kapitel dokumentiert die technische Implementierung der BuyIn Todo Sandbox, die als Plattform für das standardisierte Laborsetting dieser Studie dient. Es erläutert, wie die Brownfield-Umgebung architektonisch aufgebaut ist, um realistische Anforderungen zu simulieren. Zudem wird beschrieben, welche technischen Maßnahmen ergriffen wurden, um eine reproduzierbare Datenerhebung zu gewährleisten.

5.1. Anforderungen an die Sandbox

Für nachvollziehbare Aussagen über Vibe-Coding in diesem Kontext wird die Sandbox-Umgebung entlang der folgenden Anforderungen beschrieben:

1. **Realismus (Brownfield):** Die Codebasis ist nicht leer („Greenfield“), sondern enthält bestehende Strukturen, technische Schulden und Konventionen, die im Arbeitsprozess zu berücksichtigen sind.
2. **Komplexität:** Der Tech-Stack ist modern und hinreichend komplex, um triviale Lösungen zu vermeiden.
3. **Isolierbarkeit:** Die Umgebung kann für jeden Teilnehmenden identisch zurückgesetzt werden (Containerisierung).
4. **Messbarkeit:** Das System stellt automatisierte Ausführungsläufe bereit, um den funktionalen Fortschritt standardisiert zu erfassen.

5.2. Architektur und Tech-Stack: Kognitive Dimensionen

Die Sandbox ist als klassische Full-Stack-Webanwendung konzipiert. Die Architekturwahl hat direkte Auswirkungen auf die kognitive Belastung der Teilnehmenden, analysiert durch das Framework der *Cognitive Dimensions of Notation* (Green 1989).

5.2.1. Backend: Node.js & Express (Layered Architecture)

Das Backend basiert auf Node.js mit dem Express-Framework und folgt einer strikten Schichtenarchitektur:

- **Routes:** Definition der API-Endpunkte.
- **Controllers:** Verarbeitung der HTTP-Requests und Eingabepprüfung.
- **Services:** Kapselung der Geschäftslogik.
- **Repositories:** Datenzugriffsschicht.

Kognitive Dimension - Viscosity (Zähigkeit): Diese Trennung erhöht die *Viscosity* für Novizen, da Änderungen häufig an mehreren Stellen (Datei-Hopping) vorgenommen werden. Für die Assistenzinteraktion ist die Struktur zugleich klar segmentiert: Dateinamen und Pfade bilden Kontexthinweise, die in Prompts referenziert werden können.

5.2.2. Frontend: React & Vite

Das Frontend nutzt React mit TypeScript. Die Komponenten sind funktional aufgebaut und nutzen Hooks. **Kognitive Dimension - Hidden Dependencies:** Die Verwendung von *Custom Hooks* und *Context API* erzeugt *Hidden Dependencies*. Low-Code-Teilnehmende sehen im UI-Code nicht sofort, woher die Daten kommen. Damit wird im Setting relevant, wie Abhängigkeiten im Projektkontext identifiziert und wie in der Assistenzinteraktion geeigneter Kontext bereitgestellt wird.

5.3. Technische Funktionsweise von Copilot in der Sandbox

Zur Einordnung, warum die Sandbox Copilot fordert, ist die technische Funktionsweise relevant.

5.3.1. Prompt Construction und Context Retrieval

Copilot sendet nicht nur den Code vor dem Cursor an das Modell. Der *Copilot Proxy* in der IDE sammelt Kontext aus:

- **Jaccard Similarity:** Offene Tabs werden nach Ähnlichkeit zum aktuellen Code gescannt.
- **Semantic Search:** Vektorisierte Suche im Projekt (bei neueren Versionen).

In der Sandbox sind die Aufgaben so gestaltet, dass die Lösung oft in einer *anderen* Datei liegt (z.B. ist der Controller anzupassen, um den Service zu nutzen). Wenn der Nutzer diese Datei nicht öffnet, ist der relevante Kontext in der Assistenzinteraktion typischerweise nicht verfügbar. Dadurch wird indirekt abgebildet, in welchem Maß Nutzende relevanten Kontext bereitstellen.

5.3.2. Token Prediction und Ranking

Das Modell generiert mehrere Vorschläge (Beams) und rankt diese nach Wahrscheinlichkeit. In einer Brownfield-Umgebung ist die Wahrscheinlichkeit für „korrekten“ Code höher, wenn er dem Stil der Umgebung entspricht. High-Coder-Teilnehmende erkennen, ob ein Vorschlag den Stil bricht (z.B. `var` statt `const`); Low-Code-Teilnehmende häufig nicht.

5.4. Design der Aufgaben (Workload Grading)

Die Aufgaben sind so gestaffelt, dass sie die kognitive Last sukzessive erhöhen:

Task	Beschreibung	Kognitive Anforderung
1	Persistenz (JSON)	Verstehen des Repository-Patterns (Backend).
2	User-Accounts	Einführung neuer Entitäten, Anpassung mehrerer Layer.
3	Kategorien (UI)	Frontend-Logik, State-Management.
4	Attachments	Umgang mit Binärdaten, API-Erweiterung.
5	Kalender	Komplexe Logik, Datumsberechnung (hoher Intrinsic Load).
6	Email-Verifikation	Integration externer Konzepte (Mocking).
7	Performance	Optimierung, kein funktionales Feature (Experten-Task).
8	ICS-Export	Standard-Format, reines Mapping (Erholung).

Tabelle 5.1.: Aufgaben-Design und kognitive Anforderungen (Abk.: JSON = JavaScript Object Notation; UI = User Interface; ICS = iCalendar).

5.5. Automatisierung der Auswertung

Um die Auswertung der Ergebnisse zu standardisieren, wurde ein Auswertungs-Framework entwickelt.

5.5.1. Das Analyse-Skript (`run.js`)

Ein Node.js-Skript automatisiert die Auswertung der Lösungen. Es führt folgende Schritte aus:

1. **Ausführung:** Ausführung der Unit- und End-to-End-Läufe (Vitest, Playwright).
2. **Linting:** Analyse von Code-Style-Verletzungen mittels ESLint.
3. **Report-Generierung:** Zusammenfassung der Ergebnisse in einer JSON-Datei.

5.5.2. Grenzen der LLM-gestützten Code-Analyse

Zusätzlich zur rein funktionalen Auswertung wird ein LLM-Prompt-Template verwendet, um qualitative Aspekte des Codes einzuordnen. Dabei ist kritisch anzumerken, dass in der Literatur Halluzinationen beschrieben werden: Es werden Befunde berichtet, bei denen Fehler benannt werden, die nicht existieren, oder echte Probleme unentdeckt bleiben. Der Score dient daher nur als Indikator und wird stichprobenartig manuell nachgesehen.

5.6. Relevanz für den Unternehmenskontext (BuyIn)

Die gewählte Architektur und Methodik spiegelt reale Herausforderungen bei BuyIn wider:

- **Legacy-Integration:** Wie bei BuyIn arbeiten Entwickler häufig in gewachsenen Strukturen, die nicht perfekt dokumentiert sind. Vibe-Coding wird hier als Arbeitsmodus beschrieben, in dem „impliziter Kontext“ über iteratives Nachfragen, Zusammenfassen und Abgleichen erschlossen wird.
- **Security & Compliance:** Der Einsatz von Copilot in Enterprise-Umgebungen birgt Risiken (Datenabfluss, Lizenzverletzungen) (Pearce u. a. 2022). Die Sandbox simuliert dies durch Aufgaben, bei denen sensible Daten (User-Accounts) behandelt werden.
- **Skalierbarkeit:** Die modulare Struktur der Sandbox bildet einen Kontext, um den Einsatz von Copilot in stärker modularisierten Codebasen (z.B. größere Teams) im Kontrast zu kleinen Skript-Projekten einzuordnen.

5.7. Containerisierung und Reproduzierbarkeit

Die gesamte Umgebung ist mittels Docker containerisiert. Dies stellt sicher, dass alle Teilnehmenden – unabhängig von ihrem Betriebssystem – unter identischen Bedingungen arbeiten und technische Störfaktoren (z.B. unterschiedliche Node-Versionen) reduziert werden.

5.7.1. Reproduzierbarkeit / Artefaktzugang

Die in dieser Arbeit berichteten quantitativen Ergebnisse (Ausführungsläufe, Linting und daraus abgeleitete Scores) sind aus dem Repository heraus nachvollziehbar reproduzierbar. Neben der containerisierten Sandbox liegen die zentralen Auswerteskripte sowie die genutzten Rohdaten (Chat- und Survey-Exporte) versioniert vor.

Ablage der relevanten Artefakte.

- `backend/`, `frontend/`: Anwendungscode inkl. automatisierten Läufen und ESLint-Konfiguration.

- `evaluation/`: Ausführbare Auswerteskripte und erzeugte Ergebnisdateien (z.B. `result.json`, `result.md`).
- `thesis/chats/`: exportierte Chatverläufe mit dem KI-Assistenzsystem (`.docx`).
- `thesis/surveys/`: Survey-Export zur Kompetenz-Operationalisierung (`.xlsx`).

Minimaler Reproduktionspfad (4 Schritte).

1. Abhängigkeiten installieren: `npm ci --prefix backend`
2. Abhängigkeiten installieren: `npm ci --prefix frontend`
3. Auswertung ausführen: `node evaluation/run.js`
4. Ergebnisse einsehen: `evaluation/result.json`, `evaluation/result.md`. Zusätzlich (Detailartefakte):
 - `backend/test-results.json`
 - `frontend/test-results.json`
 - `frontend/playwright-report.json`
 - `backend/lint-results.json`
 - `frontend/lint-results.json`

Erweiterte Reproduktion (optional). Für die im Ergebnisteil verwendete Pipeline-Klassifikation kann zusätzlich `node evaluation/run_pipeline_v3.js` ausgeführt werden. Die Klassifikation basiert auf drei Evidenzstufen:

- „Attempted“ aus Chat-Signalen
- „Implemented_{static}“ aus Code-Heuristiken
- „Execution observed“ durch Integrationsläufe

Das Skript erzeugt u.a. `evaluation/task_pipeline_v3.json` und `evaluation/task_pipeline_v3.csv`. Die Survey-Zuordnung zu Branches liegt als erzeugtes Artefakt vor (`evaluation/survey_mapping.json`); die Neugenerierung via `node evaluation/map_survey.js` ist möglich. Sie kann jedoch ggf. eine Anpassung der im Skript hinterlegten Dateipfade erfordern.

Artefakt	Zweck	Erzeugt durch
evaluation/result.json	Aggregierte Task- und Lint-Ergebnisse pro Branch/Run	run.js
evaluation/result.md	Menschlich lesbare Zusammenfassung der Checks	run.js
backend/test-results.json	Backend-Testergebnisse (Jest) als JSON-Report	run.js
frontend/test-results.json	Frontend-Testergebnisse (Vitest) als JSON-Report	run.js
frontend/playwright-report.json	E2E-Testergebnisse (Playwright) als JSON-Report	run.js
evaluation/task_pipeline_v3.json	Pipeline-Klassifikation je Branch (Attempted, Implemented und Execution observed)	run_pipeline_v3.js
evaluation/survey_mapping.json	Zuordnung: Branch – Teilnehmende Person – Kompetenzgruppe	map_survey.js

Tabelle 5.2.: Zentrale Auswertungsartefakte (Dateipfade, Zweck und Erzeugung) im Repository; JSON-Reports sind maschinenlesbare Ergebnisartefakte, E2E steht für End-to-End-Tests.

6. Ergebnisse

Zur Orientierung gilt: Die Pipeline-Status *Attempted*, *Implemented_{static}* und *Execution observed* beschreiben unterschiedliche Evidenzebenen desselben Arbeitsprozesses und sind keine Qualitäts- oder Kompetenzbewertung. *Implemented_{static}* bezeichnet strukturelle Artefakte in der Codebasis; *Execution observed* ist an protokollierte Ausführungsausgaben gebunden.

6.1. Datengrundlage und Toolkontext

Dieses Unterkapitel beschreibt die Datengrundlage der Untersuchung sowie den technischen Kontext der durchgeführten *Vibe-Coding-Challenge*. Die Analyse basiert auf einem Datensatz von $N = 18$ Probanden, die im Rahmen einer kontrollierten Coding-Challenge Aufgaben zur Entwicklung einer Todo-Applikation bearbeiteten.

6.1.1. Studiensetting und Rahmenbedingungen

Die Studie wurde unter einheitlichen Rahmenbedingungen durchgeführt, um die Vergleichbarkeit der Ergebnisse zu gewährleisten:

- **Zeitlimit:** Alle Probanden hatten ein striktes Zeitlimit von 60 Minuten für die Bearbeitung der Aufgaben.
- **Tooling:** Als Entwicklungsumgebung diente Visual Studio Code mit der GitHub Copilot Extension.
- **KI-Modell:** Als zugrundeliegendes Large Language Model (LLM) wurde Claude Sonnet 4.5 verwendet. Die Interaktion erfolgte über das Chat-Interface von GitHub Copilot, wobei das Modell explizit als Backend konfiguriert war.
- **Aufgabenstellung:** Die Probanden erhielten eine Liste von 8 Aufgaben (T1–T8) mit steigender Komplexität, beginnend bei Backend-Persistenz bis hin zu komplexen Frontend-Features und Export-Funktionen.

6.1.2. Datenquellen und Artefakte

Die empirische Auswertung basiert auf drei primären Datenquellen, die eine Triangulation der Ergebnisse ermöglichen:

1. Code-Artefakte (Git-Repositories): Die finalen Implementierungen der Probanden liegen als separate Git-Banches vor. Diese Branches enthalten den vollständigen Quellcode zum Zeitpunkt des Zeitablaufs.
 - Pfad: Remote-Banches mit dem Suffix `*-vibe`.
Beispiel: `origin/nachname-vorname-vibe`.
2. Chat-Logs: Die vollständigen Interaktionsprotokolle zwischen Proband und KI-Assistenzsystem wurden exportiert. Diese Protokolle enthalten alle Prompts der Nutzer sowie die in den Exports als Antworten des KI-Assistenzsystems markierten Textsegmente (inklusive ggf. enthaltener Codeblöcke).
 - Pfad: `thesis/chats/` (Format: `.docx`).
3. Survey-Daten: Im Anschluss an die Challenge füllten die Probanden einen Fragebogen zur Selbsteinschätzung und subjektiven Wahrnehmung der KI-Interaktion aus.
 - Pfad: `thesis/surveys/`
Probanden-Einstufungsformular(Responses).xlsx.

6.1.3. Dateninventar und Vollständigkeit

Artefakt-Typ	Anzahl	Anmerkung
Git-Banches (<code>*-vibe</code>)	18	Vollständig als Remote-Banches vorhanden
Chat-Logs	18	Vollständig für alle Teilnehmer
Survey-Einträge	18	Vollständig ($N = 18$)

Tabelle 6.1.: Inventar der in der Auswertung verwendeten Datenartefakte (Git-Banches, Chat-Logs, Survey-Responses) und ihre Verfügbarkeit über alle Teilnehmenden ($N = 18$).

Messung. Erfasst wurde die Verfügbarkeit der drei Datenartefakte Git-Banches, Chat-Logs und Survey-Einträge über alle Teilnehmenden ($N = 18$). **Hauptbefund.** Für alle drei Artefakt-Typen liegt Vollständigkeit vor (jeweils 18/18). **RQ-Bezug.** Das Dateninventar ist die Grundlage für die Triangulation von Code-, Ausführungs- und Chat-Artefakten (insbesondere RQ2) sowie für die Auswertung der Pipeline-Status in Kapitel 6.

6.1.4. Zuordnung und Zusammenführung der Daten

Die Zusammenführung der Datenquellen erfolgte über eine Mapping-Tabelle, die Namensangaben aus dem Survey und die Chat-Dateinamen den entsprechenden Git-Banches zuordnet. In der Ergebnisdarstellung werden Teilnehmende ausschließlich als Teilnehmer A–R referenziert.

- **Survey ↔ Branch:** Das Survey-Feld "Branch-Name" sowie der Name des Probanden dienten als Schlüssel.
- **Chat ↔ Branch:** Die Dateinamen der Chat-Logs (z.B. Nachname_Vorname.docx) wurden normalisiert und den entsprechenden Branches zugeordnet.
Beispiel: nachname-vorname-vibe.

Die Survey-Daten enthalten zudem eine Einteilung in Kompetenzgruppen (No-Code, Low-Code, Mid-Code, High-Coder), die in der späteren Analyse als vergleichendes Merkmal herangezogen wird.

6.1.5. Reproduzierbarkeit und Auswertungswerkzeuge

Um die Ergebnisse nachvollziehbar und reproduzierbar zu gestalten, kommen automatisierte Analysewerkzeuge zum Einsatz, deren Resultate im Ordner **evaluation/** persistiert werden:

- **Statische Code-Analyse:** Einsatz von ESLint zur Analyse der Code-Qualität und Einhaltung von Standards.
- **Automatisierte Ausführung:** Ausführung der vorhandenen Suite (basierend auf Jest/Vitest), um funktionale Umsetzungsstände der Implementierungen (Task Completion) systematisch zu erfassen.
- **Chat-Parsing:** Algorithmische Auswertung der Chat-Logs zur Bestimmung von *Attempted*-Tasks und Identifikation von Prompting-Mustern.

6.1.6. Abgrenzung zu Kapitel 7

Das vorliegende Kapitel 6 beschränkt sich auf die deskriptive Darstellung der Ergebnisse und die objektive Auswertung der Daten. Eine Interpretation dieser Ergebnisse, die Diskussion möglicher Wirkzusammenhänge sowie die Beantwortung der Forschungsfragen erfolgen im anschließenden Kapitel 7 (Diskussion).

6.2. Task-Pipeline-Analyse

In diesem Abschnitt wird der Fortschritt der Probanden entlang der Aufgaben-Pipeline (T1–T5) dargestellt. Berichtete werden Häufigkeiten für Intention (Chat), Umsetzung (Code-Artefakte) und den Statusausweis *Execution observed* (Integrationsläufe).

6.2.1. Methodische Definitionen

Für die Auswertung werden drei Statuskategorien verwendet:

- **Attempted (Versucht):** Ein Task gilt als *Attempted*, wenn er im Chat-Verlauf explizit als Ziel, Anforderung oder nächster Arbeitsschritt formuliert wurde. Die bloße Existenz von Dateien ohne entsprechenden Kontext im Chat genügt nicht.
- **Implemented_{static} (Umgesetzt, Artefakt/Heuristik):**
Ein Task gilt als *Implemented_{static}*, wenn er in den Auswertungsartefakten als implementiert markiert ist (heuristische, statische Indikatoren; z.B. Feld `implemented` in `evaluation/task_pipeline_v3.json` bzw. Implementationsklassen in `evaluation/deep_code_analysis.json`).
- **Execution observed (Statusausweis):** Ein Task gilt als *Execution observed*, wenn die zugehörigen automatisierten Integrationsläufe erfolgreich durchlaufen wurden (Feld `verified` in `evaluation/task_pipeline_v3.json`).

Abgeleitete Evidenzstufe. In Kapitel 6 wird eine abgeleitete Evidenzstufe *Implemented** verwendet:

$$Implemented^* := Attempted \wedge (Implemented_{static} \vee Execution\ observed).$$

Damit gilt per Konstruktion: $Execution\ observed \subseteq Implemented^*$. *Execution observed* ist jedoch nicht notwendigerweise ein Subset von *Implemented_{static}*: *Implemented_{static}* basiert auf heuristischen, strukturellen Erkennungsregeln; unter den gegebenen Rahmenbedingungen ist es möglich, dass funktionsfähige Implementationen nicht als *Implemented_{static}* erkannt werden (z.B. abweichende Benennungen, alternative Architekturpfade oder Implementationen außerhalb der erwarteten Muster). *Execution observed* ist ein methodengebundenes Diagnose-Signal; *Implemented_{static}* ist ein Artefakt-Signal.

Hinweis zur Ausführungsmethodik. Für die Pipeline-Auswertung wurden die automatisierten Integrationsläufe (insbesondere T2, sowie in Teilen T3–T5) als API-Integrationsläufe (*Supertest* gegen Express) gestaltet. *Execution observed* bedeutet in dieser Studie explizit: „durchläuft die aktuellen Integrationsläufe“ – nicht „fachlich vollständig korrekt gelöst“.

6.2.2. Pipeline-Ergebnisse (T1–T5)

Messung. Berichtete werden je Task (T1–T5) die Häufigkeiten der Pipeline-Status *Attempted* (Chat-Signal), *Implemented** (kombinierte Evidenzstufe) und *Execution observed* (methodengebundenes Signal) über $N = 18$ Branches. **Hauptbefund.** Für T1/T2 liegt *Attempted* jeweils in 18/18 Branches vor, während *Execution observed* für T4/T5 in 0/18 Branches erreicht wird. **RQ-Bezug.** Die Statushäufigkeiten bilden die deskriptive Ergebnisbasis für RQ1 und werden in Kapitel 7 für die Einordnung der Befunde und die Pipeline-Diskussion verwendet.

Messung. Visualisiert werden die in Tabelle 6.2 berichteten Statushäufigkeiten (*Attempted*, *Implemented**, *Execution observed*) als Balken pro Task. **Hauptbefund.** Über T1–T5 liegen *Implemented** und *Execution observed* unter *Attempted* und erreichen bei T4/T5 den Wert 0. **RQ-Bezug.** Die Abbildung ist eine komprimierte Darstellung der Pipeline-Ergebnisse und ergänzt die Ergebniszusammenfassung zu RQ1 in Kapitel 7.

Task	Attempted (<i>n</i>)	Implemented* (<i>n</i>)	Execution observed (<i>n</i>)
T1: Persistenz	18 (100%)	17 (94%)	10 (56%)
T2: Auth	18 (100%)	17 (94%)	4 (22%)
T3: Kategorien	15 (83%)	13 (72%)	13 (72%)
T4: Attachments	13 (72%)	8 (44%)	0 (0%)
T5: Kalender	13 (72%)	0 (0%)	0 (0%)

Tabelle 6.2.: Task-Pipeline: Von der Intention zur testgebundenen Bestätigung ($N = 18$, n = Anzahl Branches). *Attempted* stammt aus Chat-Signalen. *Implemented** ist definiert als: *Attempted* und (*Implemented_{static}* oder *Execution observed*). *Execution observed* stammt aus V3-Integrationsläufen (V3 bezeichnet die für diese Studie definierte Integrationslauf-Konfiguration).

6.2.3. Kurzbefunde pro Task (T1–T5)

T1 (Persistenz). T1 ist in 18/18 Branches *Attempted*, in 17/18 Branches *Implemented** und in 10/18 Branches *Execution observed*.

T2 (User Accounts/Auth). T2 ist in 18/18 Branches *Attempted*, in 17/18 Branches *Implemented** und in 4/18 Branches *Execution observed*.

T3 (Kategorien). T3 ist in 15/18 Branches *Attempted*, in 13/18 Branches *Implemented** und in 13/18 Branches *Execution observed*. In `evaluation/task_pipeline_v3.json` ist T3 dabei in 7/18 Branches als `implemented` (*Implemented_{static}*) markiert.

T4 (Attachments). T4 ist in 13/18 Branches *Attempted*, in 8/18 Branches *Implemented** und in 0/18 Branches *Execution observed*.

T5 (Kalender/Multi-Day). T5 ist in 13/18 Branches *Attempted*, in 0/18 Branches *Implemented** und in 0/18 Branches *Execution observed*.

6.3. Detaillierte Lösungsmuster pro Task (T1–T5)

Dieses Unterkapitel beschreibt typische Lösungsmuster, die in den $N = 18$ *-vibeBranches für die Tasks T1–T5 beobachtet wurden. Grundlage sind Repository-Struktur und Implementationsartefakte (Backend/Frontend, Datenhaltung), Chat-basierte *Attempted*-Signale sowie Survey-Angaben (aggregiert, ohne Klarnamen). Die Darstellung ist rein deskriptiv.

Einordnung der Datenbasis (kurz)

T1 und T2 wurden im Chat durchgängig adressiert (18/18), T3 in 15/18 sowie T4/T5 in jeweils 13/18 Fällen. Entsprechend nimmt die Abdeckung der Chat-basierten *Attempted*-Signale von T1/T2 zu T4/T5 ab. In den Surveys werden Zeitdruck und Debugging am häufigsten als Hürden genannt (je 5/18), gefolgt von Aufgabenverständnis (3/18). Die Selbsteinstufung verteilt sich über No-Code (5), Low-Code (4), Mid-Code (5) und High-Coder (4).

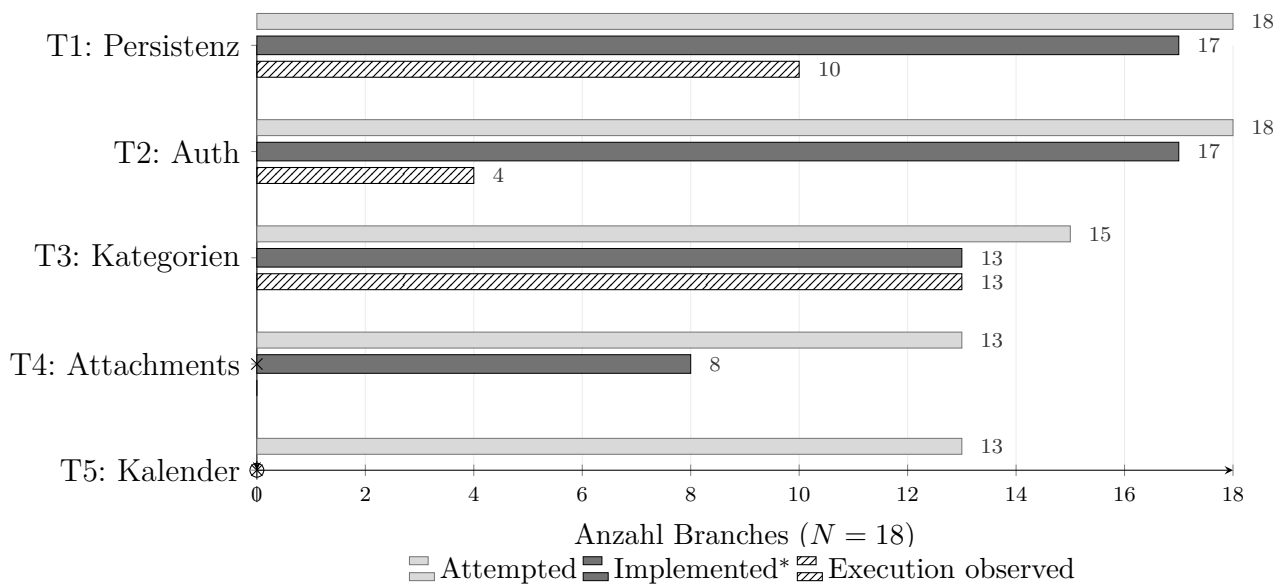


Abbildung 6.1.: Pipeline-Ergebnisse für T1–T5 ($N = 18$) als gruppierte Balken: *Attempted* (Chat-Signal), *Implemented** (definiert als: *Attempted* und (*Implemented_{static}* oder *Execution observed*)) und *Execution observed* (testgebunden: bestandene V3-Integrationsläufe). V3 bezeichnet die für diese Studie definierte Integrationslauf-Konfiguration; die Werte entsprechen Tabelle 6.2.

T1 – Persistente Todos

1. **Ziel des Tasks (kurz, technisch).** Persistente Speicherung von Todos über Prozess-Neustarts hinweg, inkl. stabilem CRUD-Verhalten (Erstellen, Lesen, Aktualisieren, Löschen) und konsistenter Identifikatoren.
2. **Typische Implementierungsvarianten (mit Häufigkeiten).** Die häufigste Variante war eine SQLite-basierte Persistenz (14/18). Daneben traten MongoDB (1/18) und PostgreSQL (1/18) sowie verbleibende Varianten wie In-Memory/sonstige Ablagen (2/18) auf. In der Code-Struktur wird häufig ein Repository-Pattern (z.B. `TodoRepository`) mit einer klaren Abgrenzung zwischen Controller/Route und Datenzugriff verwendet.
3. **Beobachtete Stabilitätsmuster (konsistent vs. inkonsistent).** Konsistent sind konsistente ID-Schemata und deterministisches CRUD. Inkonsistent sind v.a. uneinheitliche ID-Felder (`id` vs. `_id`) und inkonsistente Update-Semantik.
4. **Beobachtete Implementationsmuster.**

Häufig treten Repository-/Controller-Boilerplates und generische CRUD-Endpunkte auf. Randfälle (Eingabepfung, Fehlercodes, ID-Normalisierung) sind in den Artefakten nicht in allen Branches durchgängig konsistent umgesetzt.

5. **Beobachtete Teilumsetzungen / offene Punkte.** In einzelnen Branches bleiben Randfälle (Eingabepprüfung, Fehlercodes, ID-Normalisierung) sowie Konsistenzanforderungen (einheitliche ID-Felder, deterministische Update-Semantik) unvollständig.

T2 – Benutzerkonten / Authentifizierung

1. **Ziel des Tasks (kurz, technisch).** Nutzerregistrierung und Login mit Zugriffskontrolle auf geschützte Endpunkte (z.B. `GET /api/todos`) sowie konsistenter Authentifizierungsweitergabe (Token oder Session).
2. **Typische Implementierungsvarianten (mit Häufigkeiten).** JWT ist die häufigste Variante (11/18), gefolgt von Session/Cookie (4/18) und Hybrid-Ansätzen (1/18). Selten sind Sonderfälle (z.B. Hashing ohne klaren Token-Flow: 1/18) bzw. fehlende oder unklare Signale (1/18). Häufige Strukturmuster sind zentrale Auth-Middleware oder Kontrollen direkt im Controller.
3. **Beobachtete Stabilitätsmuster (konsistent vs. inkonsistent).** Konsistent sind eindeutige Token-/Session-Flows und konsistente Route-Guards. Inkonsistent sind inkonsistente Header-/Cookie-Nutzung und divergierende Erwartungen zwischen Frontend und Backend.
4. **Beobachtete Implementationsmuster.** Boilerplate-Flows (Register, Login, Protect) treten häufig auf. In den Artefakten finden sich sowohl Varianten mit Rollen-/Claims-Strukturen als auch minimal gehaltene Flows (z.B. ohne durchgängige Eingabepprüfung/Expiry). Häufig treten Kombinationen aus JWT und `bcrypt` auf.
5. **Beobachtete Teilumsetzungen / offene Punkte.** In einzelnen Branches sind Header-/Cookie-Nutzung und Route-Guards inkonsistent oder zwischen Frontend und Backend nicht durchgängig abgestimmt.

T3 – Kategorien

1. **Ziel des Tasks (kurz, technisch).** Erweiterung des Todo-Datenmodells um Kategorisierung und Bereitstellung über API und UI, sodass Todos kategoriebasiert erstellt, gespeichert und angezeigt werden können.
2. **Typische Implementierungsvarianten (mit Häufigkeiten).** Ein separates Kategorie-Konzept (Model/Tabelle/Collection mit Relation via `categoryId`) dominiert (14/18). Inline-Kategorien als String sind selten (1/18). In 3/18 Branches ist das Kategorie-Konzept in Struktursignalen nicht klar erkennbar.
3. **Beobachtete Stabilitätsmuster (konsistent vs. inkonsistent).** Konsistent sind konsistente Persistenz und stabile Referenzen (IDs/Relation). Inkonsistent sind rein UI-seitige Kategorien oder inkonsistente Relation/Serialisierung.

4. **Beobachtete Implementationsmuster.** Häufig treten zusätzliche **Category**-Modelle und Routen auf. In einzelnen Branches werden Kategorien und Tags terminologisch vermischt; dabei variiert die Bezeichnung, während die Datenrepräsentation nicht in allen Fällen konsistent vereinheitlicht ist.
5. **Beobachtete Teilumsetzungen / offene Punkte.** In einzelnen Branches bleiben Kategorien rein UI-seitig, oder Persistenz/Serialisierung/Relation ist nicht durchgängig konsistent umgesetzt.

T4 – Dateianhänge (Attachments)

1. **Ziel des Tasks (kurz, technisch).** Upload und Zuordnung von Dateien zu Todos (typischerweise `multipart/form-data`), Speicherung (Dateisystem oder DB) und Rückgabe verwertbarer Metadaten über die API.
2. **Typische Implementierungsvarianten (mit Häufigkeiten).** In 8/18 Branches ist ein Multer-/Upload-Ansatz erkennbar. In 10/18 Branches sind Attachments nicht oder nur indirekt oder unklar umgesetzt. Wo Upload existiert, ist Storage häufig lokal (Uploads-Ordner) oder metadatenbasiert (Dateiname/URL).
3. **Beobachtete Stabilitätsmuster (konsistent vs. inkonsistent).** Konsistent sind klare Upload-Verträge (Feldnamen/Metadaten) und stabile Zuordnung zum Todo. Inkonsistent sind uneinheitliche Response-Formate und fehlende Metadaten-Persistenz.
4. **Beobachtete Implementationsmuster.** Multer-Templates mit Boilerplate-Routen sind häufig. Daneben finden sich sowohl Varianten mit separaten Attachment-Entitäten als auch Upload-Varianten ohne stabilen Metadaten-Rückkanal.
5. **Beobachtete Teilumsetzungen / offene Punkte.** Wo Upload existiert, sind Response-Formate, Feldnamen und Metadatenpersistenz nicht in allen Branches konsistent.

T5 – Kalender / Multi-Day

1. **Ziel des Tasks (kurz, technisch).** Abbildung von Todos im Kalender, inklusive Unterstützung von Zeiträumen (Multi-Day), sowie konsistente Query-/Filter-Semantik (z.B. Overlap-Logik) zur effizienten Darstellung in Kalenderansichten.
2. **Typische Implementierungsvarianten (mit Häufigkeiten).** In den Branches sind Range-Felder als Struktursignale verbreitet (z.B. `dueEndDate` neben `dueDate`). Gleichzeitig ist T5 in den (V3-)Integrationsläufen in 0/18 Branches als *Execution observed* ausgewiesen.

3. **Beobachtete Stabilitätsmuster (konsistent vs. inkonsistent).** In den Artefakten treten zusätzliche Range-Felder häufig auf. Eine durchgängig erkennbare Range-Logik (Ende > Start) sowie eine konsistente Query-/Filter-Semantik (Overlap-Logik) ist in den Artefakten nicht in allen Branches erkennbar.
4. **Beobachtete Implementationsmuster.** Häufig treten UI-/Modell-Boilerplates (zusätzliche Datumsfelder, Kalender-Komponenten) sowie Range-Utilities auf. In den Artefakten ist der API-Vertrag (Parameter/Overlap-Regel) dabei nicht in allen Branches konsistent erkennbar.
5. **Beobachtete Teilumsetzungen / offene Punkte.** In den Artefakten sind Range-Felder und UI-Komponenten teilweise vorhanden, ohne dass eine durchgängig konsistente, durch *Execution observed* ausgewiesene End-to-End-Semantik für Overlap/Filter und API-Vertrag vorliegt.

6.4. Code-Qualität und technische Metriken

Dieses Unterkapitel beschreibt beobachtbare Aspekte der Code-Qualität über alle 18 *-vibe-Banches hinweg. Im Fokus stehen technische Messwerte und Artefakte (Linting, Typisierungs-Surrogate, Ausführungsausgänge und Strukturindikatoren), die die Wartbarkeit und Robustheit des Systems ergänzend zur rein funktionalen Korrektheit beschreiben. Die Darstellung bleibt bewusst deskriptiv. Sie bildet Messstände und Auswertungsergebnisse ab, ohne kausale Zuschreibungen oder Wirkzusammenhänge abzuleiten. Als Datenbasis dienen ausschließlich die Auswertungsartefakte:

- `evaluation/code_quality_metrics.json`
- `evaluation/data_safe/lint_results.json`
- `evaluation/data_safe/test_results.json`
- `evaluation/deep_code_analysis.json`
- `evaluation/task_pipeline_v3.json`

6.4.1. ESLint und Typisierung

Die ESLint-Ergebnisse liegen als Fehleranzahl pro Branch vor. In `evaluation/data_safe/lint_results.json` weisen 4 von 18 Branches 0 ESLint-Fehler aus, während 14 von 18 Branches 1 ESLint-Fehler ausweisen. Eine Aufschlüsselung nach ESLint-Regelkategorien (z. B. *no-unused-vars*, *no-explicit-any*) ist in den Artefakten nicht enthalten und wurde daher nicht ausgewertet.

Als Surrogat für Typisierungsstrenge und potenziellen “Escape” aus dem Typsystem wird die Nutzung von `any` ausgewiesen (`anyUsage` in `evaluation/code_quality_metrics.json`). Über alle Branches liegt `anyUsage` zwischen 2 und 42 (Median: 8.5).

Zusätzlich wird die Verwendung von `console` als Indikator für Debug-Ausgaben berichtet (`consoleUsage`). Diese liegt zwischen 4 und 19 (Median: 6). Spezifische Vorkommen von `TODO/FIXME` oder “temporären” Debug-Endpunkten werden in den vorliegenden Metriken nicht erfasst.

6.4.2. Ausführungsausgänge und Stabilität

Die Ausführungsdaten liegen in zwei Formen vor: (a) eine Aggregation von Pass/Fail-Zählwerten je Branch in `evaluation/deep_code_analysis.json` sowie (b) Jest-JSON-Zusammenfassungen je Branch in `evaluation/data_safe/test_results.json`. In `evaluation/deep_code_analysis.json` reichen die pro Branch gezählten `test_pass`-Werte von 0 bis 14 und die `test_fail`-Werte von 9 bis 37. 7 von 18 Branches weisen `test_pass=0` aus. 11 von 18 Branches weisen mindestens einen erfolgreichen Lauf aus. Die Jest-JSON-Ausgaben spiegeln dieses Bild in Form von Summen. Beispiele sind `numFailedTestSuites`, `numFailedTests` und `numPassedTests`. Eine normalisierte Kategorisierung der Fehlertypen (z. B. nach “Auth/HTTP-Status”, “Schema”, “Datenpersistenz”) ist in den Artefakten nicht enthalten.

Für die Aufgabenfortschritts-Metrik wird zusätzlich `evaluation/task_pipeline_v3.json` herangezogen. Für den dort ausgewiesenen Statusausweis (`verified`) ergeben sich über alle Branches: T1 ist in 10/18 Branches als `Execution observed` ausgewiesen, T2 in 4/18, T4 in 0/18 und T5 in 0/18. Für T5 wird in `evaluation/task_pipeline_v3.json` außerdem `implemented=0/18` ausgewiesen. Bei T3 sind in `evaluation/task_pipeline_v3.json` `verified=13/18` und `implemented=7/18` vermerkt. In den Auswertungsartefakten ist `implemented` dabei als heuristisches, statisches Signal (*Implemented_{static}*) zu verstehen, während `verified` einen methodengebundenen Statusausweis abbildet. Entsprechend ist unter den gegebenen Rahmenbedingungen möglich, dass `verified` empirisch größer als `implemented` ausfällt. Für eine konsistente Zusammenführung wird diese Differenz über die in Abschnitt 6.2 definierte Evidenzstufe *Implemented** ($Attempted \wedge Implemented_{static} \vee Execution\ observed$) abgebildet.

6.4.3. Architektur- und Strukturindikatoren

Als strukturbezogene Indikatoren werden in `evaluation/deep_code_analysis.json` für einzelne Aufgaben Implementationsklassen sowie Persistenzvarianten ausgewiesen. Für T1 (Persistenz) werden unterschiedliche Backends beobachtet: 14/18 Branches sind als *sqlite* klassifiziert, jeweils 1/18 als *mongo*, *postgres*, *json-file* und *memory/other*. Für T2 (Auth) werden 17/18 Branches als *implemented* und 1/18 als *none* klassifiziert. Für T3 (Kategorien) sind 7/18 Branches als *implemented* und 11/18 als *none* klassifiziert. Für T4 (Attachments) sind 8/18 als *implemented* und 10/18 als *none* klassifiziert. Für T5 (Kalendar) ist in `evaluation/deep_code_analysis.json` über alle 18 Branches hinweg *none* ausgewiesen, konsistent zu `evaluation/task_pipeline_v3.json` (`implemented=0/18`, `verified=0/18`).

Die Codeumfangs-Metrik `tsFileCount` (`evaluation/code_quality_metrics.json`) liegt über alle Branches zwischen 43 und 80 TypeScript-Dateien (Median: 54) und liefert

damit einen groben, rein quantitativen Indikator für Codebasis-Variation.

6.4.4. Zusammenhänge mit Task-Fortschritt

Für eine rein deskriptive Gegenüberstellung wird die Anzahl als `implemented` markierter Tasks pro Branch aus `evaluation/task_pipeline_v3.json` genutzt. Die Branches verteilen sich dabei auf 1 bis 4 als `implemented` markierte Tasks (T5 ist in allen Fällen nicht implementiert). In einer gruppierten Gegenüberstellung variieren die Mittelwerte von `anyUsage` und `consoleUsage` zwischen den Gruppen der `implemented`-Taskanzahl (z. B. `anyUsage` im Mittel 8.8 bei 2 implementierten Tasks vs. 12.4 bei 3 vs. 16.7 bei 4). Analog ergeben sich in der Gegenüberstellung nach ESLint-Fehleranzahl (0 vs. 1 Fehler laut `evaluation/data_safe/lint_results.json`) Unterschiede in den Medianen von `consoleUsage` (4.5 vs. 7.5) und `anyUsage` (7.5 vs. 9). Diese Darstellung ist aufgrund der kleinen Fallzahl bei 0 Fehlern (4 Branches) als rein deskriptiv zu lesen.

Darüber hinaus enthalten die vorliegenden Artefakte keine Kennzahlen zu Testabdeckung in Prozent, keine Komplexitätsmaße (z. B. zyklomatische Komplexität), keine TypeScript-Compilerfehlerstatistiken, keine Performance-/Bundle-Metriken und keine konsolidierten Frontend-Lighthouse-Ergebnisse. Aus diesem Grund werden hier keine weitergehenden, metrisch gestützten Aussagen über Abdeckung, Komplexität oder Laufzeitverhalten getroffen.

6.4.5. Kurzfazit: Beobachtete Qualitätsmerkmale

- ESLint: 4/18 Branches ohne ESLint-Fehler, 14/18 mit genau einem Fehler (keine Regelkategorien verfügbar).
- Typisierung/Debug-Surrogate: `anyUsage` variiert stark (2–42), `consoleUsage` liegt zwischen 4 und 19.
- Ausführungsausgänge: Ausgänge sind heterogen (`test_pass` 0–14 und `test_fail` 9–37). 7/18 Branches weisen `test_pass=0` aus.
- Statusausweis (Pipeline v3): T1 ist in 10/18 Branches als Execution observed ausgewiesen, T2 in 4/18. T4 und T5 sind in 0/18 als Execution observed ausgewiesen.
- Architekturindikatoren: Persistenz- und Feature-Varianten sind in den Artefakten ausgewiesen (mehrere Persistenztypen, T2 überwiegend implementiert, T5 in allen Branches nicht implementiert).

6.5. Chat-Interaktionsmuster

Dieses Unterkapitel beschreibt beobachtbare Muster der Interaktion zwischen den Teilnehmenden (Teilnehmer A–R) und dem im Experiment eingesetzten Tool *GitHub Copilot* mit dem zugrundeliegenden LLM *Claude Sonnet 4.5*. Als Datenbasis dienen ausschließlich die 18 Chat-Exports in `thesis/chats/` (.docx) sowie der Task-Status

aus `evaluation/task_pipeline_v3.json`. Ein separates, maschinell generiertes Chat-Metrik-Artefakt liegt nicht vor. Alle berichteten Metriken wurden direkt aus den Chat-Exports extrahiert. Die Zuordnung Chat $\leftrightarrow$ Branch erfolgt über die im Evaluationskapitel beschriebene Normalisierung der Chat-Dateinamen und deren Mapping auf Branches. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet. Eine qualitative Inhaltsbewertung der Antworten erfolgt in diesem Unterkapitel nicht.

6.5.1. Umfang und Intensität der Interaktion

Messdefinition. Für alle 18 Chats wurde die Wortanzahl als whitespace-tokenisierte Wortzählung auf dem aus `.docx` extrahierten Text bestimmt. Wortanzahl, Zyklenzahl und Anzahl der Nutzerbeiträge sind in diesem Abschnitt als Intensitätsindikatoren operationalisiert und werden nicht als Leistungs- oder Effizienzmaße interpretiert.

Ergebnis. Die Wortanzahl pro Chat liegt zwischen 4046 und 26833 Wörtern (Median: 7137.5, Quartile: Q1=5330.5, Q3=8862.25).

Messdefinition. Die Anzahl der Prompt-Antwort-Zyklen wurde als Sequenzpaar aus einem Nutzerbeitrag (in den Exports typischerweise markiert durch `You said:` oder einen Namenspräfix `X:`) und der unmittelbar folgenden Modellantwort (in den Exports u. a. markiert durch `GitHub Copilot:` oder `ChatGPT said:`) gezählt. Die genannten Sprecherkennzeichnungen werden ausschließlich zur Segmentierung in Nutzerbeiträge und Modellantworten verwendet.

Ergebnis. Über alle Chats liegt die Anzahl der Prompt-Antwort-Zyklen zwischen 12 und 45 (Median: 16.5, Quartile: Q1=14.25, Q3=19.75).

Messdefinition. Als zusätzlicher Intensitätsindikator wurde die Anzahl der erkannten Nutzerbeiträge pro Chat bestimmt.

Ergebnis. Diese liegt zwischen 53 und 777 Nutzerbeiträgen (Median: 144). Damit werden in den Chat-Exports sowohl Fälle mit wenigen Dutzend Nutzerbeiträgen als auch Fälle mit mehreren hundert Nutzerbeiträgen beobachtet.

6.5.2. Struktur der Prompts

Messdefinition. Für die formale Beschreibung der Prompt-Struktur wurden alle 3390 erkannten Nutzerbeiträge aggregiert ausgewertet. Die folgenden Merkmale wurden regelbasiert bestimmt. Die Kategorien sind nicht disjunkt, ein Nutzerbeitrag kann mehrere Merkmale gleichzeitig aufweisen.

Ergebnis. Ein-Satz-Prompts (operationalisiert als Nutzerbeiträge mit höchstens einem Satzendzeichen `.?!)` treten in 1901 von 3390 Fällen auf (56.1%). Strukturierte Mehrpunkt-Prompts (operationalisiert über Aufzählungs-/Schrittmuster, z. B. `•` oder nummerierte Listen sowie Schlüsselwörter wie `To do/Schritt`) treten in 471 von 3390 Fällen auf (13.9%).

Kontextangaben (operationalisiert über Kontextmarker wie `repo`, `backend`, `frontend`, `docker` oder `workspace`) treten in 1555 von 3390 Fällen auf (45.9%). Explizite Anforderungen/Constraints (operationalisiert über modale/negierende Marker wie `muss/must/do not/ausschließlich`) treten in 307 von 3390 Fällen auf (9.1%). Schrittanweisungen

(operationalisiert über Marker wie **Schritt/first/zuerst** oder nummerierte Sequenzen) treten in 382 von 3390 Fällen auf (11.3%).

Nachforderungen bzw. erneute Anweisungen (operationalisiert über Wiederholungs-/Umformulierungsmarker wie **nochmal/anders/rewrite**) treten in 67 von 3390 Fällen auf (2.0%).

6.5.3. Debugging-Interaktionen und Iterationsmuster

Messdefinition. Debugging-bezogene Nutzerbeiträge wurden über Vorkommen typischer Fehlermarker (z. B. **error**, **exception**, **failed**, **TypeError**, **TS...**, **eslint**) erfasst. Die folgenden Iterations- und Wiederholungsmetriken beschreiben ausschließlich Textmuster in den Nutzerbeiträgen. Daraus werden keine Aussagen über Emotionen, Kompetenz oder Motivation abgeleitet.

Ergebnis. Insgesamt enthalten 296 von 3390 Nutzerbeiträgen solche Fehlermarker (8.7%).

Messdefinition. Als *Iteration* wurde eine Folge von Nutzerbeiträgen operationalisiert, die jeweils Fehlermarker enthalten (aufeinanderfolgende Korrekturversuche im selben Chatabschnitt).

Ergebnis. Über alle Chats hinweg wurden 254 solche Iterationssequenzen gezählt. Die maximale Länge aufeinanderfolgender Fehlermarker-Nutzerbeiträge in einem einzelnen Chat liegt zwischen 1 und 9 (Median: 2, Maximum: 9).

Messdefinition. Als einfache Form von *Wiederholung* wurde zusätzlich erfasst, ob zwei aufeinanderfolgende Fehlermarker-Nutzerbeiträge eine identische normalisierte Fehlersignatur (erste erkannte Error-/Exception-Zeile) enthalten.

Ergebnis. Über alle Chats hinweg wurden 23 unmittelbare Wiederholungen identischer Fehlersignaturen gezählt.

Messdefinition. Als Abbruch-/Wechselindikator wurden Task-Sprünge in den Nutzerbeiträgen über Task-Mentions (**Task 1-8**, **Aufgabe 1-8**, **T1-T8**) erfasst. Task-Wechsel werden in diesem Unterkapitel als Strukturmerkmal der Interaktion berichtet und nicht bewertet.

Ergebnis. 148 von 3390 Nutzerbeiträgen enthalten mindestens eine Task-Mention (4.4%). In 8 von 18 Chats wird mindestens ein Wechsel der zuerst genannten Task-Nummer beobachtet. Über alle Chats hinweg wurden 74 solche Task-Wechsel gezählt.

6.5.4. Beobachtete Antwortmuster in den Modellantworten (Claude Sonnet 4.5)

Messdefinition. Für die Modellantworten wurden 335 erkannte Antworten aggregiert ausgewertet. Als Modellantworten werden Textsegmente erfasst, die in den Exports als Antwort des KI-Assistenzsystems markiert sind (z. B. durch **GitHub Copilot:** oder **ChatGPT said:**). Diese Kennzeichnungen werden ausschließlich zur Segmentierung verwendet.

Ergebnis. Codeblöcke in der Modellantwort (operationalisiert über das Vorkommen von Markdown-Fences "``") treten in 122 von 335 Modellantworten auf (36.4%). Rückfragen bzw. Frageformen (operationalisiert über ein abschließendes Fragezeichen oder Formulierungen wie *Would you like/Can you*) treten in 21 von 335 Modellantworten auf (6.3%).

Tooling-/Aktionsformulierungen (operationalisiert über Marker wie *Ran terminal command* oder *Replace String in File*) treten in 41 von 335 Modellantworten auf (12.2%). Architektur-/Strukturbegriffe (operationalisiert über Schlüsselwörter wie *controller, service, repository* u.a.) treten in 152 von 335 Modellantworten auf (45.4%).

6.5.5. Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)

Messdefinition. Für eine rein deskriptive Gegenüberstellung wird der Task-Fortschritt je Branch aus *evaluation/task_pipeline_v3.json* neben die Interaktionsmetriken gestellt. Dabei wird *implemented* als heuristisches, statisches Artefakt-Signal verstanden; dies wird im Folgenden als *Implemented_{static}* bezeichnet. *verified* bildet den Statusausweis *Execution observed* ab. Die folgenden Aussagen beschreiben Ko-Vorkommen innerhalb derselben Branches; es werden keine Wirk- oder Kausalzusammenhänge abgeleitet.

Ergebnis. Über die 18 Branches hinweg liegen die *implemented*-Taskanzahlen (d.h. *Implemented_{static}*) zwischen 1 und 4 (T5 ist in *evaluation/task_pipeline_v3.json* in allen Branches nicht implementiert).

In der Gruppe mit 3 *implemented*-Tasks (n=8; *Implemented_{static}*) liegt die Wortanzahl zwischen 4863 und 26833 (Median: 8865) und die Zyklenzahl zwischen 16 und 45 (Median: 18). Die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). In der Gruppe mit 4 *implemented*-Tasks (n=3; *Implemented_{static}*) liegt die Wortanzahl zwischen 4764 und 6663 (Median: 4919) und die Zyklenzahl zwischen 14 und 21 (Median: 14). Die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2).

Dieses Unterkapitel beschreibt ausschließlich beobachtbare Interaktionsmuster und deren Messwerte. Es enthält keine Effizienz- oder Leistungsbewertung, keine Kompetenzzuschreibung, keine psychologischen Deutungen und keine Modellbewertung. Die Interpretation dieser Muster sowie deren Einordnung erfolgt ausschließlich in Kapitel 7.

6.6. Triangulation der Datenquellen

Dieses Unterkapitel stellt die in Abschnitt 6.2 bis 6.5 berichteten Ergebnisse sowie Survey-Merkmale nebeneinander. Ziel ist eine rein deskriptive Triangulation auf Ebene der 18 Branches (Teilnehmer A–R): Pipeline-Fortschritt (*attempted, implemented* und *verified*), beobachtete Strukturindikatoren der Implementationen, technische Metriken, Interaktionsmetriken aus den Chat-Exports sowie Survey-Angaben. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet. Daraus werden

keine Wirk- oder Kausalzusammenhänge abgeleitet. Die folgenden Abschnitte ordnen diese Datenquellen paarweise und gruppenweise nebeneinander und berichten Muster und Spannweiten. **Ergebnis.** Über alle Branches hinweg ist bei T1 *sqlite* in 14/18 Fällen ausgewiesen. Daneben treten *mongo* (1/18), *postgres* (1/18), *json-file* (1/18) und *memory/other* (1/18) auf. Bei Gruppierung nach **implemented**-Taskanzahl ergibt sich folgende Verteilung der Persistenzvarianten: bei 2 implementierten Tasks (n=6) *sqlite* in 5/6 Fällen und *postgres* in 1/6, bei 3 implementierten Tasks (n=8) *sqlite* in 7/8 und *mongo* in 1/8, bei 4 implementierten Tasks (n=3) *sqlite* in 2/3 und *json-file* in 1/3. Der einzelne Branch mit 1 implementiertem Task ist *memory/other*.

Messdefinition. Die Triangulation erfolgt pro Branch als gemeinsame Analyseinheit. Der Pipeline-Fortschritt wird aus `evaluation/task_pipeline_v3.json` als Anzahl der pro Branch als **implemented** bzw. **verified** markierten Tasks (T1–T5) abgeleitet. Dabei wird **implemented** als heuristisches, statisches Artefakt-Signal (*Implemented_{static}*) verstanden. Die in Abschnitt 6.2 eingeführte Evidenzstufe *Implemented** wird in diesem Unterkapitel nicht als Gruppierungsvariable verwendet. Als ergänzende Strukturindikatoren (z. B. Persistenzvariante) und Test-Summen (`test_pass/test_fail`) wird `evaluation/deep_code_analysis.json` genutzt. Technische Metriken wie `tsFileCount`, `anyUsage` und `consoleUsage` stammen aus `evaluation/code_quality_metrics.json`. Survey-Merkmale (Kompetenzgruppe und Hürdenkategorie) werden aus dem Survey-Export (`thesis/surveys/Probanden-Einstufungsformular(Responses).xlsx`) übernommen und über das Mapping `evaluation/survey_mapping.json` den Branches zugeordnet. Chat-bezogene Metriken werden in diesem Unterkapitel nicht neu berechnet, sondern als die in Abschnitt 6.5 berichteten Werte (Extraktion aus `thesis/chats/`) übernommen.

Ergebnis. Die Branches verteilen sich auf 1 bis 4 als **implemented** markierte Tasks (Verteilung: 1 → 1 Branch, 2 → 6 Branches, 3 → 8 Branches, 4 → 3 Branches). Für **verified** ergeben sich 0 bis 2 verifizierte Tasks je Branch (Verteilung: 0 → 4 Branches, 1 → 1 Branch, 2 → 13 Branches). T5 ist in `evaluation/task_pipeline_v3.json` in allen Branches nicht implementiert und nicht verifiziert.

6.6.1. Pipeline-Fortschritt und Implementationsmuster (Strukturindikatoren)

Messdefinition. Als Stellvertreter für ausgewählte Implementationsstrategien werden Strukturindikatoren aus `evaluation/deep_code_analysis.json` herangezogen. Für T1 betrifft dies die klassifizierte Persistenzvariante (z. B. *sqlite*, *mongo*). Für T2–T4 werden Implementationsklassen (**implemented/none**) berichtet.

Ergebnis. Über alle Branches hinweg dominiert bei T1 *sqlite* (14/18). Daneben treten *mongo* (1/18), *postgres* (1/18), *json-file* (1/18) und *memory/other* (1/18) auf. Bei Gruppierung nach **implemented**-Taskanzahl ergibt sich folgende Verteilung der Persistenzvarianten: bei 2 implementierten Tasks (n=6) *sqlite* in 5/6 Fällen und *postgres* in 1/6, bei 3 implementierten Tasks (n=8) *sqlite* in 7/8 und *mongo* in 1/8, bei 4 implementierten Tasks (n=3) *sqlite* in 2/3 und *json-file* in 1/3. Der einzelne Branch mit 1 implementiertem Task ist *memory/other*.

Für T2 wird in 17/18 Branches `implemented` ausgewiesen (Quelle: `evaluation/deep_code_analysis.json`). Konsistent dazu liegen in `evaluation/task_pipeline_v3.json` `implemented` für T2 in 17/18 Branches vor. Für T3 sind 7/18 Branches in `evaluation/task_pipeline_v3.json` als `implemented` (*Implemented_{static}*) markiert. Gleichzeitig ist T3 in 13/18 Branches `verified`. Für T4 sind 8/18 Branches als `implemented` markiert. `verified` ist bei T4 in allen Branches 0/18.

6.6.2. Pipeline-Fortschritt und technische Metriken

Messdefinition. Für eine deskriptive Gegenüberstellung werden pro Branch technische Metriken aus `evaluation/code_quality_metrics.json` (`tsFileCount`, `anyUsage`, `consoleUsage`) sowie Test-Summen aus `evaluation/deep_code_analysis.json` (`test_pass`, `test_fail`) neben die Anzahl `implemented` Tasks gestellt.

Ergebnis. Über alle 18 Branches liegt `tsFileCount` zwischen 43 und 80 Dateien. `anyUsage` liegt zwischen 2 und 42 und `consoleUsage` zwischen 4 und 19. Gruppiert nach `implemented`-Taskanzahl ergeben sich folgende Spannweiten und Mediane:

- 1 implementierter Task (n=1): `anyUsage`=6, `consoleUsage`=6, `tsFileCount`=43, `test_pass`=14, `test_fail`=12.
- 2 implementierte Tasks (n=6): `anyUsage` 2–30 (Median: 4), `consoleUsage` 4–17 (Median: 4.5), `tsFileCount` 43–63 (Median: 56.5), `test_pass` 0–14 (Median: 12) und `test_fail` 9–37 (Median: 11).
- 3 implementierte Tasks (n=8): `anyUsage` 3–42 (Median: 8.5), `consoleUsage` 4–19 (Median: 10), `tsFileCount` 49–80 (Median: 54), `test_pass` 0–12 (Median: 0) und `test_fail` 12–28 (Median: 15.5).
- 4 implementierte Tasks (n=3): `anyUsage` 15–19 (Median: 16), `consoleUsage` 4–9 (Median: 6), `tsFileCount` 47–74 (Median: 57), `test_pass` 0–13 (Median: 12) und `test_fail` 10–13 (Median: 12).

Diese Gegenüberstellung beschreibt ausschließlich gemeinsame Verteilungen innerhalb derselben Branches.

6.6.3. Pipeline-Fortschritt und Chat-Interaktionsmetriken

Messdefinition. Als Chat-Metriken werden die in Abschnitt 6.5 definierten und berichteten Intensitäts- und Interaktionsindikatoren (Wortanzahl, Prompt-Antwort-Zyklen, Nutzerbeiträge, Fehlermarker-Iterationen) herangezogen.

Für die Triangulation werden diese Werte neben die `implemented`- und `verified`-Taskanzahl aus `evaluation/task_pipeline_v3.json` gestellt.

Ergebnis. In Abschnitt 6.5 wird bereits eine gruppierte Gegenüberstellung nach `implemented`-Taskanzahl berichtet. Dort liegen (a) in der Gruppe mit 2 implementierten Tasks (n=6) Wortanzahl und Zyklenzahl zwischen 4046–9190 Wörtern (Median: 6449) bzw. 12–20 Zyklen (Median: 14.5), (b) in der Gruppe mit 3 implementierten Tasks (n=8)

zwischen 4863–26833 Wörtern (Median: 8865) bzw. 16–45 Zyklen (Median: 18) und (c) in der Gruppe mit 4 implementierten Tasks (n=3) zwischen 4764–6663 Wörtern (Median: 4919) bzw. 14–21 Zyklen (Median: 14).

Über alle Gruppen hinweg liegt die verifizierte Taskanzahl in `evaluation/task_pipeline_v3.json` je Branch zwischen 0 und 2.

6.6.4. Pipeline-Fortschritt und Survey-Merkmale

Messdefinition. Aus dem Survey werden (a) die Kompetenzgruppe (No-Code, Low-Code, Mid-Code, High-Coder) und (b) eine primäre Hürdenkategorie (Survey-Feld D) als kategoriale Merkmale herangezogen. Beide Merkmale werden ausschließlich als Gruppierungs- bzw. Häufigkeitsvariablen genutzt.

Ergebnis. Die Kompetenzgruppen verteilen sich auf No-Code (5/18), Low-Code (4/18), Mid-Code (5/18) und High-Coder (4/18).

Kompetenzgruppe	1 impl.	2 impl.	3 impl.	4 impl.
No-Code (n=5)	0	2	2	1
Low-Code (n=4)	1	1	2	0
Mid-Code (n=5)	0	1	2	2
High-Coder (n=4)	0	2	2	0

Tabelle 6.3.: Kompetenzgruppe (Survey) vs. Anzahl `implemented` ($Implemented_{static}$) Tasks pro Branch (`evaluation/task_pipeline_v3.json`, $N = 18$).

Messung. Gegenübergestellt wird die Kompetenzgruppe (Survey) und die Anzahl als `implemented` markierter Tasks pro Branch ($Implemented_{static}$) für $N = 18$. **Hauptbefund.** Vier `implemented`-Tasks treten nur in No-Code (1) und Mid-Code (2) auf, während Low-Code und High-Coder diese Kategorie nicht erreichen. **RQ-Bezug.** Die Häufigkeitstabelle liefert eine deskriptive Zuordnung von Kompetenzgruppen zu Artefaktfortschritt und adressiert damit RQ1.

Als primäre Hürdenkategorien (Survey-Feld D) treten am häufigsten Zeitdruck (5/18) und Debugging von Fehlern (5/18) auf, gefolgt von Aufgabenverständnis (3/18). In der Gruppe mit 2 implementierten Tasks (n=6) ist Zeitdruck 2/6-mal und Debugging 1/6-mal als primäre Hürde genannt. In der Gruppe mit 3 implementierten Tasks (n=8) ist Debugging 3/8-mal und Zeitdruck 2/8-mal als primäre Hürde genannt. In der Gruppe mit 4 implementierten Tasks (n=3) treten Zeitdruck, Debugging und Aufgabenverständnis jeweils in 1/3 Fällen als primäre Hürde auf.

6.6.5. Abgrenzung zu Kapitel 7

Dieses Unterkapitel stellt ausschließlich Ko-Vorkommen und Verteilungen über mehrere Datenquellen hinweg dar. Es enthält keine Leistungs- oder Effizienzbewertung, keine Kompetenzzuschreibung und keine kausale Interpretation. Die Diskussion dieser Muster erfolgt ausschließlich in Kapitel 7.

6.7. Zusammenfassung der Ergebnisse

Kapitel 6 dokumentiert die Ergebnisse der Auswertung und stellt beobachtete Ausprägungen und Verteilungen dar, ohne diese einzuordnen oder zu erklären. Die Ergebnisdarstellung basiert auf mehreren Datenquellen (Pipeline-Status `attempted`, `implemented` und `verified`, Code- und Ausführungsartefakte, Chat-Exports sowie Survey-Merkmale) und bleibt durchgängig deskriptiv. Im Folgenden werden die zentralen Befundlinien systematisch zusammengeführt.

Pipeline-Ergebnisse entlang der Tasks (T1–T5). Über die Tasks hinweg werden abnehmende Statushäufigkeiten entlang der Pipeline-Stufen berichtet: Attempted-Markierungen sind in der Breite vorhanden, während die Anteile der als Implemented* (Evidenzstufe) bzw. Execution observed ausgewiesenen Ergebnisse über T1–T5 abnehmen. Dabei variiert der Umsetzungsstand zwischen den Branches. In der Gegenüberstellung der Aufgaben werden zwei Task-Gruppen unterschieden:

- **Tasks T1/T2.** T1 ist in den Branches in unterschiedlichen Persistenzvarianten umgesetzt, wobei eine Variante am häufigsten auftritt und mehrere Alternativen vereinzelt vorkommen. T2 wird in der Mehrzahl der Branches als Implemented* ausgewiesen.
- **Tasks T3–T5.** Bei T3 und T4 nehmen die Implemented_{static}-Ausweise ab. Zugleich werden bei einzelnen Tasks Diskrepanzen zwischen Implemented_{static} (heuristische Artefakt-Markierung) und Execution observed berichtet. T5 bleibt in den Artefakten durchgängig ohne Implemented_{static}- und ohne Execution observed-Ausweise.

Über alle Tasks hinweg gilt: Execution observed, Implemented_{static} und Attempted sind empirisch unterscheidbare Statuskategorien und fallen nicht notwendigerweise zusammen. Für die aggregierte Darstellung wird Implemented* ($\text{Attempted} \wedge (\text{Implemented}_{\text{static}} \vee \text{Execution observed})$) als konsistente Evidenzstufe genutzt. Kapitel 6 beschreibt diese Statusausweise als Ergebnisvariablen der Pipeline und nutzt sie als gemeinsame Referenz, um Code-, Ausführungs- und Chat-Artefakte pro Branch in Beziehung zu setzen.

Technische Qualität (aggregiert, ohne Kausalannahmen). Die in Kapitel 6 berichteten Qualitäts- und Technikindikatoren fallen insgesamt heterogen über die Branches hinweg aus. Dazu gehören (a) Unterschiede im Linting-Status und in der Häufigkeit typischer TypeScript-Surrogate (z. B. Any-Nutzung), (b) Spannweiten bei strukturbezogenen Metriken (z. B. Anzahl TypeScript-Dateien) sowie (c) variierende Testausgänge bis hin zu Konstellationen mit Fehlanteilen. Diese Befunde werden in Kapitel 6 als gemeinsames Auftreten von Pipeline-Fortschritt und technischen Merkmalen beschrieben. Es wird keine Korrelation oder Wirkbeziehung behauptet. Zusätzlich wird berichtet, dass sich technische Ausprägungen nicht auf einen einheitlichen "Projektzustand" reduzieren lassen: Branches mit ähnlichem Pipeline-Fortschritt können unterschiedliche Muster in Linting-, Typisierung- und Testindikatoren aufweisen. Zugleich werden die Messgrenzen betont: Die verwendeten Indikatoren ersetzen keine Coverage-, Komplexitäts- oder

Performance-Metriken und erlauben entsprechend keine Aussagen zu nicht gemessenen Qualitätsdimensionen.

Chat-Interaktionsmuster (kompakt). Die Chat-Analysen berichten große Spannweiten bei der Interaktionsintensität, insbesondere bei Wortanzahl und Prompt-Antwort-Zyklen. Über die Branches hinweg treten kurze, wenig strukturierte Prompts sowie detaillierte Mehrpunkt-Prompts auf. Beide Formen kommen im selben Datensatz nebeneinander vor. Als wiederkehrendes Strukturmerkmal werden Debugging-Iterationen berichtet (z. B. wiederholte Fehlermarker und erneute Anpassungsversuche), ohne dass daraus eine normative Einordnung abgeleitet wird. Neben Intensitätsmaßen werden in Kapitel 6 auch Muster in der Interaktionsstruktur beschrieben, etwa wiederkehrende Task-Referenzen, Wechsel zwischen Aufgaben sowie Tooling-/Aktionsformulierungen im Verlauf der Chats.

Triangulation auf Meta-Ebene. Die Triangulation berichtet, dass unterschiedliche Datenquellen teils kongruente, teils divergente Signale liefern: Pipeline-Status, technische Indikatoren, Testausgänge, Chat-Merkmale und Survey-Merkmale können für dieselben Branches zusammenpassen oder auseinanderlaufen. Insbesondere wird berichtet, dass $\text{Execution observed} \neq \text{Implemented}_{\text{static}} \neq \text{Attempted}$ und dass Abweichungen zwischen Statusausweisen, technischen Beobachtungen und Interaktionsmustern empirisch auftreten. Auch Survey-Merkmale werden als kategoriale Kontextvariablen einbezogen (Selbsteinstufungsgruppen und benannte Hürdenkategorien), ohne dass daraus individuelle Zuschreibungen oder Bewertungen abgeleitet werden. In der Gesamtschau ergeben sich damit nebeneinander stehende Ergebnislinien, die Gemeinsamkeiten und Unterschiede zwischen Branches beschreiben, ohne diese zu erklären. Kapitel 6 berichtet diese Abweichungen, ohne sie zu erklären.

Abgrenzung zu Kapitel 7. Kapitel 6 beschreibt, *was* beobachtet wurde. Kapitel 7 diskutiert die Einordnung der Befunde im Kontext der Forschungsfragen.

7. Diskussion

7.1. Zentrale Erkenntnisse dieser Studie

- **Bedingung: Brownfield & Timebox (60 Minuten).** **Beobachtung:** Das Statusmuster ist konsistent: T1/T2 werden nahezu durchgängig adressiert und artefaktseitig umgesetzt, während der Statusausweis *Execution observed* bei T4/T5 ausbleibt. **Einordnung:** In diesem Setting wird beobachtet, dass frühe Implementationschritte häufiger erreicht werden; Integrations- und Abgleichsarbeit bleibt ein Engpass.
- **Bedingung: Mehrere Evidenzebenen.** **Beobachtung:** Chat-Intention (*Attempted*), Artefakt-Signale (*Implemented_{static}*) und automatisierte Statussignale (*Execution observed*) fallen empirisch auseinander (z.B. T3: *Execution observed* kann größer als *Implemented_{static}* sein). **Einordnung:** Die Befunde werden daher nicht über Einzelmetriken (Chat-Intensität, Lint, ein Statussignal) allein eingeordnet.
- **Bedingung: Kompetenzgruppen als Analyseperspektiven.** **Beobachtung:** Kompetenzgruppen unterscheiden sich in der Verteilung von *Implemented_{static}*-Ausweisen (vgl. Tabelle 6.3); gleichzeitig liefern die Befunde keine Grundlage für eine einfache Rangordnung nach *Execution observed*-Ausweisen. **Einordnung:** Der Befund lässt sich im Hinblick auf gruppenspezifische Risiken einordnen, ohne Kompetenz als Leistungsranking zu instrumentalisieren.
- **Bedingung: Integrationsaufgaben (T3–T5).** **Beobachtung:** Bei steigender Kopplung (Schema → API → UI) sinken *Implemented** und *Execution observed* deutlich. **Einordnung:** Der Befund lässt sich im Hinblick auf Schnittstellenverträge, Ausführung im Systemkontext und Review gegenüber bloßer Codeproduktion einordnen.
- **Bedingung: Chat als Arbeitsoberfläche.** **Beobachtung:** Interaktionsintensität variiert stark und liefert in diesem Datensatz kein eindeutiges Effizienz- oder Qualitätssignal. **Einordnung:** Prozessqualität lässt sich über nachvollziehbare Artefakte (CI-Ausgaben, diff-basierte Reviews, Logging) beschreiben.

7.2. Ziel und Abgrenzung der Diskussion

Kapitel 6 hat die Ergebnisse ausschließlich deskriptiv berichtet; Kapitel 7 ordnet diese Befunde im Licht des in Kapitel 2 eingeführten theoretischen Rahmens ein. Ziel ist es, die

Forschungsfragen anhand der berichteten Befundlinien zu beantworten und theoriegeleitete Perspektiven als Strukturierung aufzugreifen. Kapitel 7 führt keine neuen Daten ein und berechnet keine zusätzlichen Metriken; kausale Effekte werden nicht beansprucht.

Der methodische Beitrag dieser Arbeit liegt in der dokumentierten, artefaktgebundenen Auswertungslogik, die Pipeline-Status, Code- und Ausführungsartefakte sowie Chat- und Survey-Artefakte systematisch zusammenführt und dadurch eine nachvollziehbare, deskriptive Ergebnisdarstellung im kontrollierten Brownfield-Setting ermöglicht. Diese Vorgehensweise trägt dazu bei, Intention (über Interaktions- und Bearbeitungssignale), Implementationsspuren und *Execution observed*-Ausweise getrennt zu behandeln und in einer gemeinsamen Referenzlogik zu berichten. Der Beitrag ist auf die im Manuskript beschriebenen Rahmenbedingungen (u. a. Sandbox, Timebox, Datenquellen und Auswertungsregeln) begrenzt und beansprucht keine Verallgemeinerung über diesen Geltungsbereich hinaus.

Methodischer Rahmen (Kausalität, Gruppenvergleiche). Die Studie ist als kontrollierte, artefaktbasierte Beobachtungsstudie angelegt; Randomisierung, eine Manipulation von Bedingungen und prozessnahe Messungen zur Identifikation von Wirkzusammenhängen liegen nicht vor. Aus diesem Grund werden die Befunde als Muster unter den in Kapitel 6 dokumentierten Bedingungen diskutiert. Auf inferenzstatistische Gruppenvergleiche wird verzichtet, da die Fallzahlen pro Kompetenzgruppe klein sind und potenzielle Konfundierungen nicht belastbar kontrolliert werden.

Geltungsbereich der Ergebnisse. Die diskutierten Befunde beziehen sich auf *interaktive* KI-Assistenzsysteme im Vibe-Coding-Kontext, angewendet in einer Brownfield-Sandbox auf ein bestehendes Codeartefakt. Sie gelten für eine zeitlich begrenzte Aufgabenbearbeitung unter den in Kapitel 6 beschriebenen Rahmenbedingungen. Nicht Gegenstand sind Greenfield-Projekte, langfristige Produktentwicklung sowie autonome Agenten mit eigener Aufgabenplanung und eigenständiger Ziel- und Teilzielbildung.

Eine zentrale Linse der Diskussion sind die vier Kompetenzgruppen No-Code, Low-Code, Mid-Code und High-Coder. Diese Gruppen werden als gleichwertige Analyseperspektiven behandelt. Die Gruppenzuordnung beruht auf einer pragmatischen, auditierbaren Pre-Survey-Heuristik (Tabelle 3.1) und beansprucht keine psychometrische Fundierung. Aussagen beziehen sich auf aggregierte Gruppenmuster oder auf einzelne Branches als Analyseeinheiten; eine Leistungsbewertung oder Rangordnung einzelner Personen erfolgt nicht.

Group comparability & confounders. Zentrale Rahmenbedingungen sind konstant gehalten (einheitliche IDE „VS Code“, identische Sandbox und Aufgabenstellung, identisches Zeitlimit, sowie dasselbe KI-Assistenzsystem aus GitHub Copilot und Claude Sonnet 4.5). Die beobachteten Unterschiede können damit als kompetenzabhängige Nutzungstypen diskutiert werden; alternative Erklärungen (u.a. Berufserfahrung, Domänenroutine, Debugging- und Abgleichroutinen, Risikotoleranz) bleiben möglich. Entsprechend

werden Unterschiede als theoriegeleitete Einordnungen diskutiert, ohne gruppenstabile Leistungsrangfolgen oder kausale Effekte zu behaupten.

7.3. Beantwortung der Forschungsfragen

RQ1: Kompetenz und Pipeline-Status

Antwortsatz (gewichtet). Unter den Bedingungen dieser Studie unterscheiden sich Kompetenzgruppen in den Verteilungen von *Implemented_{static}* und *Implemented**; ein eindeutiges, gruppenstabiles Muster in *Execution observed* ergibt sich nicht.

Begründung (Befunde). (1) In Kapitel 6 wird *Implemented_{static}* nach Kompetenzgruppen als Häufigkeitstabelle berichtet (Tabelle 6.3). Darin treten Branches mit vier als **implemented** markierten Tasks in Mid-Code häufiger auf als in High-Coder, während No-Code und Low-Code dazwischen liegen (vgl. Kapitel 6). (2) *Execution observed* ist als methodisch enger Statusausweis gefasst; Kapitel 6 berichtet hierfür vor allem taskbezogene Engpässe (vgl. Kapitel 6). (3) Eine gruppierte Auswertung von *Execution observed* wird in Kapitel 6 nicht als eigene Tabelle berichtet; entsprechend wird hier kein gruppenstabiles Muster behauptet.

Geltungsbereich / Limitation. Die Aussage bezieht sich auf Branches als Analyseinheiten, Timebox (60 Minuten) und die in Kapitel 6 dokumentierte Messmethodik. Sie ist deskriptiv; ein kausaler Einfluss von Kompetenz wird nicht beansprucht.

RQ2: Rolle von Code- und Ausführungsartefakten

Antwortsatz (gewichtet). Code- und Ausführungsartefakte sind für die Ergebniseinordnung zentral, weil sie die Trennung zwischen sichtbaren Implementationsspuren und *Execution observed*-Ausweisen operationalisieren und Chat-Signale methodisch einordnen.

Begründung (Befunde). (1) *Execution observed* ist messgebunden und kann unabhängig von heuristischen Artefaktsignalen ausfallen (vgl. Kapitel 6). (2) Für T4/T5 wird berichtet, dass *Attempted* und teils *Implemented** auftreten können, ohne dass *Execution observed* ausgewiesen wird (vgl. Kapitel 6). (3) Technische Surrogate (Lint/**anyUsage**/CI-Summen) variieren innerhalb ähnlicher Pipeline-Stände und ersetzen keine Verhaltensbeobachtung.

Geltungsbereich / Limitation. *Execution observed* bedeutet in dieser Studie: Der Status wird in den Auswertungsartefakten als **verified** ausgewiesen; er ersetzt keine vollständige fachliche Abdeckung und ist an die Messdefinition gebunden.

RQ3: Chat-Interaktionsmuster

Antwortsatz (gewichtet). Chat-Muster sind als Prozesssignale relevant, liefern allein jedoch kein belastbares Ergebnismaß; ihre Aussagekraft entsteht erst in Kombination mit Pipeline-Status sowie Code- und Ausführungsartefakten.

Begründung (Befunde). (1) Wortanzahl, Zyklenzahl und Nutzerbeiträge variieren stark (Kapitel 6.5), ohne dass daraus ein eindeutiges Muster für *Execution observed* folgt. (2) Debugging-Marker und Iterationssequenzen treten als wiederkehrendes Strukturmerkmal auf; sie beschreiben Interaktion, nicht Ergebnisqualität. (3) Chat-basierte *Attempted*-Signale und *Execution observed*-Ausweise fallen nicht notwendigerweise zusammen (z.B. T3; vgl. Kapitel 6), was die Notwendigkeit der Triangulation unterstreicht.

Geltungsbereich / Limitation. Die Chat-Metriken beruhen auf regelbasierten Extraktionen aus Exporten; sie erfassen keine IDE-Aktionen (Ausführungsläufe, Kontextbereitstellung) und damit nur einen Ausschnitt des Arbeitsprozesses.

Zur begrifflichen Trennung wird – wie in Kapitel 1 eingeführt – zwischen Assistenzumgebung (Tool), Modell (LLM) und *KI-Assistenzsystem* als funktionaler Einheit unterschieden.

7.4. Rückbindung an theoriegeleitete Perspektiven

Die Perspektiven wurden im Forschungsdesign als theoriegeleitete Deutungsfolien formuliert, wie Kompetenz und Arbeitsmodus mit dem KI-Assistenzsystem zusammenwirken. Im Folgenden wird jede Perspektive anhand der in Kapitel 6 berichteten Befundlinien diskutiert. Dabei werden die vier Kompetenzgruppen No-Code, Low-Code, Mid-Code und High-Coder als Analyseperspektiven verwendet. Nicht jede Perspektive zielt in gleicher Weise auf alle Gruppen.

7.4.1. P1: Mid-Code als möglicher Sweet Spot

Einordnung: gemischt.

Stärkstes Pro-Argument. In Kapitel 6 weist Mid-Code in der Häufigkeitstabelle zu *Implemented_{static}* mehr Branches mit hohem artefaktseitigem Fortschritt aus als High-Coder (vgl. Kapitel 6, Tabelle 6.3), was als deskriptiver Häufigkeitsunterschied zu lesen ist und nicht als Kompetenz- oder Leistungsurteil.

Stärkstes Contra-Argument. Für *Execution observed* wird in Kapitel 6 kein gruppierter Vergleich berichtet; gleichzeitig bleibt *Execution observed* bei T4/T5 in allen Branches aus (vgl. Kapitel 6). Ein klarer „Sweet Spot“ in Richtung *Execution observed*-Ausweisen lässt sich damit nicht ausweisen.

Finale Einordnung. Im Material wird eine höhere Häufigkeit artefaktseitigen Fortschritts bei Mid-Code beobachtet; zugleich zeigt sich kein eindeutiges Sweet-Spot-Muster in *Execution observed*-Ausweisen.

7.4.2. P2: Abgleich-Paradoxon bei Low-Code

Einordnung: gemischt.

Stärkstes Pro-Argument. In Kapitel 6 weist die Häufigkeitstabelle aus, dass Low-Code im Vergleich zu Mid-Code weniger Branches mit vier als **implemented** markierten Tasks aufweist und zugleich der einzige Branch mit nur einem **implemented**-Task in

Low-Code liegt (vgl. Kapitel 6, Tabelle 6.3), was als deskriptiver Häufigkeitsunterschied zu lesen ist und nicht als Kompetenz- oder Leistungsurteil.

Stärkstes Contra-Argument. Die Artefakte enthalten keine direkten Messungen von Abgleichroutinen (z.B. Ausführungsläufe pro Person), und Kapitel 6 berichtet *Execution observed* nicht gruppiert nach Kompetenz. Damit kann ein spezifisches gruppenbezogenes „Abgleich-Paradoxon“ nicht als Muster ausgewiesen werden.

Finale Einordnung. Im Material tritt ein geringerer artefaktseitiger Fortschritt von No-/Low-Code gegenüber Mid-Code auf; ob dies durch ein spezifisches „Abgleich-Paradoxon“ (Overtrust/fehlende Kontrolle) geprägt ist, bleibt mit den vorliegenden Prozessdaten offen.

7.4.3. P3: Skepsis und Qualitätsfokus bei High-Coder

Einordnung: gemischt.

Stärkstes Pro-Argument. High-Coder weist in Kapitel 6 keine Branches mit vier als **implemented** markierten Tasks aus, während Mid-Code diese Kategorie erreicht (vgl. Kapitel 6, Tabelle 6.3). Dieser Befund ist mit der Perspektive insofern vereinbar, als ein stärker selektives Vorgehen mit geringerem als **implemented** erfasstem Artefaktfortschritt einhergehen kann.

Stärkstes Contra-Argument. Ein „Qualitätsfokus“ im Sinne höherer *Execution observed*-Ausweise wird in Kapitel 6 nicht als gruppiertes *Execution observed*-Muster berichtet. Die vorliegenden Artefakte erlauben daher keine direkte Beobachtung, ob High-Coder systematisch stärker abgleichen oder zuverlässiger *Execution observed* erreichen.

Finale Einordnung. Im Material tritt auf, dass High-Coder weniger häufig heuristisch als „implementiert“ erfasst werden, ohne dass dies in *Execution observed* als Nachteil sichtbar wird; ein spezifischer Qualitätsfokus bleibt ohne prozessnahe Review-/Ausführungsdaten nicht direkt beobachtbar.

7.5. Kompetenzabhängige Interaktionsmuster

Kapitel 6 berichtet, dass Branches sich nicht nur im Umsetzungsstand unterscheiden, sondern auch in der Art der Interaktion, in der Struktur der Prompts und in der Häufigkeit von Debugging-Sequenzen. Diese Variation kann im Licht von Kapitel 2 als Ausdruck unterschiedlicher Kompetenzlagen eingeordnet werden. Die folgenden Deutungen sind explizit theoriegeleitet und formulieren keinen kausalen Anspruch. Sie beschreiben Nutzungstypen und Analyseperspektiven, nicht Qualitätsstufen oder individuelle Leistungsfähigkeit. Zentral ist dabei nicht das Tool selbst, da alle Teilnehmenden mit derselben Assistenzumgebung (GitHub Copilot) und derselben Modellkonfiguration gearbeitet haben. In dieser Einordnung ist vielmehr relevant, welche kognitiven Ressourcen und welche mentalen Modelle zur Verfügung stehen, um Vorschläge zu kontrollieren, zu integrieren und bei Bedarf zu korrigieren.

7.5.1. No-Code

Die Befunde aus Kapitel 6 werden so gelesen, dass im vorliegenden Setting der Zugang zu umsetzungsnahen Artefakten in den beobachteten Durchläufen erleichtert erscheint. Gleichzeitig erscheinen Abgleich und Integration im bestehenden System in den beobachteten Durchläufen häufig anspruchsvoll. Im Rahmen von Kapitel 2 lässt sich dies als Vibe-Coding-Arbeitsmodus einordnen, in dem in den beobachteten Durchläufen prozedurale Chunks aus dem KI-Assistenzsystem übernommen wurden, die fehlende Produktionsregeln überbrücken. Gleichzeitig kann ohne tragfähige Schemata der Intrinsic Load der Kontrolle und Integration höher ausfallen. Damit ist es im Sinne von Kapitel 2 plausibel, dass System 2 situativ nicht in dem Maß aktiviert wird, das für semantischen Abgleich erforderlich wäre, obwohl die Oberfläche plausibel wirkt. Diese Einordnung bleibt theoriegeleitet, da Kapitel 6 keine direkten Prozessmessungen des Abgleichs auf individueller Ebene enthält.

7.5.2. Low-Code

Kapitel 6 berichtet, dass Low-Code sichtbare Implementationsartefakte erzeugt, während *Execution observed* bei stärker integrierten Aufgaben nicht durchgängig ausgewiesen wird. Im Rahmen von Kapitel 2 kann dies als Verhältnis zwischen Vertrauen und Kontrolle eingeordnet werden. Dabei lässt sich im vorliegenden Setting beschreiben, dass das KI-Assistenzsystem in den beobachteten Durchläufen als Beschleuniger im Implementationsprozess genutzt wurde, während semantische Korrektheit im Systemkontext schwerer zu kontrollieren bleibt. Wenn Abgleichroutinen nicht etabliert sind, ist in dieser theoretischen Lesart plausibel, dass unkalibriertes Vertrauen häufiger auftritt, ohne dass dies aus den Artefakten als gruppenspezifischer Effekt gemessen werden kann. Die in Kapitel 6 berichteten Strategiemuster, etwa tutorial-nahe Implementationsflüsse und Boilerplate, sind mit einem stärker template-basierten Arbeitsmodus vereinbar. Diese Einordnung bleibt theoriegeleitet, da Kapitel 6 Abgleichhandlungen nicht gruppenspezifisch und prozessnah erfasst.

7.5.3. Mid-Code

Kapitel 6 berichtet, dass bei Mid-Code sowohl Umsetzungsartefakte als auch Iterationen der Nachjustierung und des Abgleichs beobachtbar sind. Im Rahmen von Kapitel 2 kann dies als Kompetenzlage eingeordnet werden, die zielorientierte Zerlegung und aktive Kontrolle kombiniert. Damit wird im vorliegenden Setting die Nutzung des KI-Assistenzsystems eher als Werkzeug in einem kontrollierten Problemlöseprozess eingeordnet. Die in Kapitel 6 berichtete Vielfalt an Prompt-Formen ist als funktionale Variation plausibel, in der kurze Prompts Beschleunigung und strukturierte Prompts Spezifikation und Kontextabgleich unterstützen. Debugging-Iterationen sind dabei als normaler Bestandteil iterativer Integrationsarbeit zu lesen, nicht als Effizienzindikator. Diese Einordnung bleibt vorsichtig, da Kapitel 6 keine direkten Messungen von Review- und Abgleichtiefe enthält.

7.5.4. High-Coder

Kapitel 6 berichtet, dass unter Zeitlimit bei anspruchsvollen Integrationsaufgaben *Execution observed* nicht durchgängig ausgewiesen wird. Im Rahmen von Kapitel 2 kann dies für High-Coder als Nutzungstyp eingeordnet werden. In dieser Einordnung wird im vorliegenden Setting die Nutzung des KI-Assistenzsystems eher für Boilerplate und Exploration beschrieben, während zentrale Entscheidungen und Integrationslogik stärker manuell kontrolliert werden. In dieser Perspektive können intensive Kontrolle und konservative Integrationsentscheidungen mit geringerem sichtbarem Fortschritt koexistieren, ohne dass daraus unmittelbar Rückschlüsse auf Qualität oder Kompetenz abzuleiten wären. Diese Einordnung bleibt vorsichtig, da Kapitel 6 keine direkte Messung von Review-Tiefe oder Kontrollhandlungen enthält.

7.6. Interpretation der Pipeline-Muster

Die Pipeline-Kategorien *Attempted*, *Implemented_{static}* und *Execution observed* strukturieren die Befunde in Kapitel 6 als drei Status-Ebenen; für eine konsistente Aggregation wird dort zusätzlich *Implemented** als abgeleitete Evidenzstufe verwendet. Die folgenden Deutungen binden diese deskriptiven Status-Diskrepanzen explizit an die in Kapitel 2 eingeführten Konzepte (u.a. Cognitive Load, Automation Bias) zurück. Für die Diskussion ist entscheidend, dass diese Kategorien unterschiedliche Aspekte desselben Arbeitsprozesses repräsentieren. In der Notation gilt: *Attempted* $\neq$ *Implemented_{static}* $\neq$ *Execution observed*. *Attempted* beschreibt Intention und Planbildung auf der Chat-Ebene. *Implemented_{static}* beschreibt aufgabenspezifische Artefakte in der Codebasis (White-Box). *Execution observed* beschreibt einen methodisch gebundenen, von außen beobachtbaren Statusausweis (Black-Box). *Execution observed* ist damit methodengebunden und als Integrationsindikator zu lesen, nicht als Leistungs- oder Qualitätsmaß. Aus Kapitel 6 folgt, dass diese Ebenen nicht zwangsläufig zusammenfallen.

7.6.1. Attempted ist nicht gleich Implemented_{static}

Die beobachtete Differenz zwischen *Attempted* und *Implemented_{static}* ist plausibel als Integrationsphänomen einzuordnen. Insbesondere Aufgaben, die mehrere Systemteile koppeln, gehen mit zusätzlichem Koordinationsaufwand einher. Im Rahmen von Kapitel 2 kann dies über Cognitive Load erklärt werden. Die Aufgabe reduziert sich nicht auf das Erzeugen von Code, sondern umfasst das Verstehen des bestehenden Kontexts, das Abstimmen von Datenmodellen und Schnittstellen sowie das Beheben von Fehlanpassungen. Dieser Aufwand dürfte den Intrinsic Load erhöhen. Im vorliegenden Setting wurden in den beobachteten Durchläufen Syntax- und Boilerplate-Generierungen durch das KI-Assistenzsystem genutzt; dies lässt sich im Rahmen von Kapitel 2 als Senkung von Extraneous Load einordnen. In den beobachteten Durchläufen blieb die semantische Konsistenzarbeit im Systemkontext jedoch weiterhin erforderlich.

7.6.2. Implemented_{static} ist nicht gleich Execution observed

Die Diskrepanz zwischen Implemented_{static} und Execution observed ist nicht als Qualitätsurteil über einzelne Branches zu lesen. Kapitel 6 betont, dass Execution observed ein Diagnosesignal innerhalb der eingesetzten Ausführungsmethodik ist. Für konfigurationsensitive Aufgaben kann Execution observed stark von Details des Endpunktvertrags und den Annahmen der Integrationsläufe abhängen. Ein fehlendes Execution observed-Signal kann daher bedeuten, dass eine Lösung heterogen, nur teilweise integriert oder nicht exakt methodenkonform ist. Es kann auch bedeuten, dass die Integrationsläufe eine bestimmte Implementationsform voraussetzen, die nicht mit allen plausiblen Lösungen kompatibel ist.

7.6.3. Zur Einordnung von Execution observed (0 Vorkommen)

Kapitel 6 berichtet für bestimmte Integrationsaufgaben, dass Execution observed ausbleibt. Im Kontext dieser Studie ist das als erwartbares Ergebnis eines strikten Zeitlimits und hoher Schnittstellenkomplexität einzuordnen. Gerade in einer Brownfield-Umgebung dürfte der Aufwand durch bestehende Konventionen, vorhandene Datenmodelle und implizite Vertragsannahmen steigen. Dies kann mit erhöhter kognitiver Belastung und Debugging-Aufwand einhergehen. Unter Automation Bias Perspektive ist zudem plausibel, dass scheinbar fertige Artefakte in einzelnen Fällen zu früh als ausreichend akzeptiert werden. Die Abwesenheit von Execution observed wird in dieser Arbeit daher nicht als Synonym für fehlendes Verständnis interpretiert, sondern kann als Signal für Integrationskosten und Abgleichengpässe gelesen werden. Execution observed (0 Vorkommen) wird damit als methoden- und settinggebundenes Ergebnis unter Zeitlimit und Brownfield-Integrationskomplexität diskutiert.

7.7. Einordnung der Chat-Interaktionsmuster

Die Chat-Ergebnisse aus Kapitel 6 zeigen starke Variation der Interaktionsintensität und wiederkehrende Muster von Debugging-Sequenzen. Kapitel 6 ordnet diese Muster nicht ein. Für die Diskussion sind drei Punkte zentral.

7.7.1. Warum Chat-Intensität kein Effizienzmaß ist

Eine hohe Interaktionsintensität kann mehrere Bedeutungen haben. Sie kann auf Exploration, auf unklare Anforderungen, auf Fehlanpassungen an den Kontext oder auf konsequenten Abgleich und Nachjustierung hinweisen. Kapitel 2 betont, dass Vibe-Coding den Schwerpunkt vom Schreiben zum Lesen und Kontrollieren verschiebt. Damit kann ein längerer Chat-Verlauf auch Ausdruck von Kontrolle und Review sein. Umgekehrt kann ein kurzer Chat-Verlauf sowohl effiziente Beschleunigung als auch unkritische Übernahme bedeuten. Ohne zusätzliche Prozessdaten, etwa Messungen von Ausführungsläufen oder IDE-Interaktionen, ist eine Effizienzdeutung nicht abgesichert.

7.7.2. Warum Debugging-Loops normal sind

Kapitel 6 berichtet Debugging-Iterationen als wiederkehrendes Strukturmerkmal. Im Rahmen von Kapitel 2 ist dies erwartbar. Mixed-Initiative Interaction bedeutet, dass Vorschläge, Rückfragen und Korrekturen in Schleifen organisiert sind. Zudem kann die Arbeit in einem bestehenden System die Wahrscheinlichkeit von Konfigurations- und Schnittstellenfehlern erhöhen. Debugging-Sequenzen sind daher als normaler Bestandteil von Integrationsarbeit einzuordnen. Sie sind nicht per se ein Indikator für geringe Kompetenz oder geringe Qualität.

7.7.3. Explorative und zielgerichtete Interaktion

Kapitel 2 unterscheidet Interaktionsmodi wie Exploration und Beschleunigung. Kapitel 6 berichtet sowohl kurze, wenig strukturierte Prompts als auch strukturierte, mehrpunktige Prompts. Diese Koexistenz ist als situativ wechselnder Arbeitsmodus plausibel. Exploration kann genutzt werden, um APIs, Projektkonventionen oder mögliche Lösungswege zu verstehen. Beschleunigung kann genutzt werden, um bekannte Muster schnell zu implementieren. Der Wechsel zwischen diesen Modi ist in einer zeitlich begrenzten Aufgabenbearbeitung plausibel.

7.8. Einordnung von Nutzung und Auswertung

Die folgenden Abschnitte fassen Einordnungen zusammen, die an die in Kapitel 6 berichteten Befunde anschließen. Sie betreffen die Einordnung von Nutzungstypen im Organisationskontext sowie die Einordnung der Auswertung von KI-Coding-Tools. Sie knüpfen an die in Kapitel 2 eingeführten Konzepte an.

7.8.1. Einordnung im Organisationskontext

Im vorliegenden Setting war die Bereitstellung eines KI-Tools allein nicht hinreichend, um die in Kapitel 6 sichtbaren Unterschiede in Nutzungstypen zu nivellieren. Die Befunde sprechen dafür, dass Kompetenz den Umgang mit Delegation, Kontrolle und Abgleich mitprägt. In dieser Einordnung erscheinen organisatorische Einbettung und Qualifizierung als Kontextfaktoren für die beobachteten Prozessunterschiede. Im Fokus stehen dabei u.a. Kompetenzaufbau zu Abgleich, Integrationsdenken und Schnittstellenverträgen sowie explizite Erwartungen an Review und Integrationsarbeit.

7.8.2. Einordnung der Auswertung von KI-Coding-Tools

Kapitel 6 berichtet, dass einzelne Signale wie Ausführungsausgänge, Linting oder Interaktionsintensität isoliert schwer interpretierbar sind. Die Befunde zeigen, dass Interpretationen je nach gemeinsamer Betrachtung mehrerer Perspektiven variieren. Insbesondere ist die Unterscheidung zwischen Intention, Implementationsartefakt und *Execution observed*-Ausweisen zentral.

Ebenso ist der Kompetenzkontext für die Einordnung der Artefakte relevant, weil aggregierte Durchschnittseffekte Nutzungstypen überdecken können.

7.8.3. Abgrenzung zu Marketing-Narrativen

In der öffentlichen Darstellung werden häufig generische Produktivitätsgewinne hervorgehoben. Die in Kapitel 6 berichteten Muster legen dagegen eine differenzierte Einordnung entlang von Integrationsaufwand, Abgleichbarkeit und Kompetenzkontext nahe. Im Rahmen von Kapitel 2 lässt sich im vorliegenden Setting die in Kapitel 6 berichtete Nutzung von KI-Assistenzsystemen als Reduktion von Extraneous Load und damit als Beschleunigung einzelner Umsetzungsschritte einordnen. Gleichzeitig verschieben sich Anforderungen hin zu Review, semantischem Abgleich und methodengebundener Absicherung. Ohne diese Prozessanteile ist im Sinne von Kapitel 2 unkalibriertes Vertrauen plausibel, ohne dass dies kausal begründet wäre. Damit ist der Nutzen nicht als automatische Eigenschaft eines Tools zu verstehen, sondern als Ergebnis einer arbeitsorganisatorischen Einbettung, die Kompetenzlagen und Integrationskosten berücksichtigt.

7.9. Limitationen und Ausblick

Die Studie ist in ihrer Aussagekraft durch mehrere Faktoren begrenzt. Erstens ist die Stichprobengröße begrenzt, wodurch gruppenspezifische Effekte nur eingeschränkt trennscharf beobachtbar sind. Zweitens setzt das strikte Zeitlimit den Fokus auf frühe Integrationsschritte und kann unvollständige Stabilisierung begünstigen. Drittens wurde GitHub Copilot als Assistenzumgebung in Kombination mit dem Modell Claude Sonnet 4.5 betrachtet, wodurch sich keine Aussagen zur Generalisierbarkeit über andere KI-Assistenzsysteme oder Modelle ableiten lassen. Viertens liegt keine Langzeitbeobachtung vor, sodass Lern- und Adaptationseffekte über Wochen oder Monate nicht erfasst werden.

Fünftens sind die Ergebnisse an den konkreten Erhebungszeitraum gebunden. Änderungen an Modellen oder Tools nach diesem Zeitraum sind in den vorliegenden Daten nicht abgebildet und wurden entsprechend nicht berücksichtigt.

Kapitel 8 fasst die zentralen Erkenntnisse zusammen und leitet daraus die abschließenden Antworten auf die Forschungsfragen ab.

8. Fazit und Schlussfolgerungen

8.1. Ziel und Einordnung des Kapitels

Kapitel 8 schließt die Arbeit ab, indem die in Kapitel 6 berichteten Ergebnisse zusammengeführt und die in Kapitel 7 diskutierten Einordnungen in prägnante Schlussfolgerungen überführt werden. Im Unterschied zu Kapitel 6 (deskriptiv) und Kapitel 7 (theoriegeleitet) dient dieses Kapitel der abschließenden Synthese und der expliziten Beantwortung der Forschungsfragen. Es werden keine neuen Daten eingeführt und keine zusätzlichen Metriken berechnet.

Übergeordnetes Ziel der Arbeit war die Auswertung eines interaktiven KI-Assistenzsystems. Diese erfolgte im Vibe-Coding-Kontext in einer kontrollierten Brownfield-Sandbox. Im Studiensetup wurde – konsistent zur Begriffsklärung in Kapitel 1 – zwischen Assistenzumgebung (Tool), Modell (LLM) und KI-Assistenzsystem als funktionaler Einheit unterschieden; betrachtet wurde GitHub Copilot als Assistenzumgebung im VS-Code-Kontext.

8.2. Beantwortung der Forschungsfragen

8.2.1. Forschungsfrage 1: Wie hängt die Entwicklerkompetenz mit dem Erreichen der Pipeline-Status Attempted, Implementedstatic und Execution observed zusammen?

Kurzantwort. Attempted und Implementedstatic sind über die Kompetenzgruppen hinweg als Artefaktspuren in den Auswertungsartefakten ausgewiesen; Execution observed als methodisch gebundener Statusausweis wird am seltensten erreicht.

Verdichtete Begründung. Kapitel 6 trennt die Ebenen Attempted, Implementedstatic und Execution observed. Attempted steht für Intention (Chat-Ebene), Implementedstatic für White-Box-Artefakte (Codebasis), Execution observed für einen methodisch gebundenen Statusausweis und ist damit methodengebunden. Für die aggregierte Darstellung wird in Kapitel 6 zusätzlich Implemented* als abgeleitete Evidenzstufe verwendet. Kompetenz dient dabei als Analyseperspektive auf unterschiedliche Integrations- und Abgleichsmodi, nicht als Leistungsbewertung. Die Befunde aus Kapitel 6 und die Einordnung in Kapitel 7 werden so gelesen, dass Implementationsfortschritt nicht notwendigerweise in Execution observed-Ausweise übergeht.

8.2.2. Forschungsfrage 2: Welche Rolle spielen Code- und Ausführungsartefakte für die Einordnung der Arbeitsergebnisse eines KI-Assistenzsystems?

Kurzantwort. Code- und Ausführungsartefakte sind zentral, weil sie die Trennung zwischen sichtbarer Implementierung (Implementedstatic) und Execution observed-Ausweisen operationalisieren und Chat-Signale methodisch ergänzen.

Verdichtete Begründung. Die Artefaktanalyse macht Implementationsfortschritt über Quellcodeänderungen, Surrogat-Signale (z.B. Linting) und strukturelle Spuren in der Codebasis nachvollziehbar. Messgebundene Ergebnisse bilden beobachtbares Verhalten unter den gegebenen Bedingungen ab und sind damit zentral für Execution observed-Ausweise im Systemkontext. Die Pipeline-Logik bildet damit eine nachvollziehbare Trennung von Intention, Implementationsartefakt und Execution observed-Ausweisen ab (Kapitel 6) und wird in Kapitel 7 methodisch eingeordnet. Kompetenz ergänzt diese Sicht als Analyseperspektive, weil sich Artefaktspuren nicht sinnvoll in einem einzelnen, gruppenunabhängigen Signal verdichten lassen.

8.2.3. Forschungsfrage 3: Welche Chat-Interaktionsmuster treten im Vibe-Coding-Kontext auf, und wie sind sie für die Auswertung einzuordnen?

Kurzantwort. Im Vibe-Coding-Kontext variieren Chat-Interaktionsmuster deutlich; ihre Intensität ist weder als Effizienzmaß noch als Qualitätsmaß geeignet. Für die Auswertung sind Chat-Muster nur im Zusammenspiel mit Pipeline-Status sowie Code- und Ausführungsartefakten aussagekräftig.

Verdichtete Begründung. Die Chat-Artefakte zeigen unterschiedliche Prompt-Strukturen, wechselnde Interaktionsmodi und wiederkehrende Debugging-Schleifen. Diese Muster werden in Kapitel 7 entlang der Interaktionsmodi Exploration und Beschleunigung eingeordnet; beide Modi treten im Material auf. Eine hohe Interaktionsintensität kann dabei gleichermaßen aus Abgleich, aus Fehlanpassungen an den Systemkontext oder aus iterativer Integration resultieren. Entsprechend ist die Einordnung von Chat-Mustern an die Pipeline-Logik zu koppeln. Attempted bildet die Intention ab, Implementedstatic die resultierenden Artefakte und Execution observed den methodisch gebundenen Statusausweis im System. Die Kompetenzgruppen liefern hierfür den Kontext als Analyseperspektiven auf Nutzungsformen, ohne dass daraus eine Einordnung einzelner Personen abgeleitet wird.

8.3. Zentrale Erkenntnisse der Arbeit

Die Arbeit verdichtet sich in folgenden Befundlinien.

- Kompetenz als Perspektive auf Nutzungstypen: Unterschiede zeigen sich in Nutzungsformen des KI-Assistenzsystems, nicht als Wertung von Personen oder

Gruppen.

- Trennung von Intention, Artefakt und Execution observed-Ausweisen: Attempted, Implementedstatic und Execution observed sind unterschiedliche Ebenen und fallen nicht notwendigerweise zusammen.
- Abgleich und Integration als Engpass: Sichtbare Implementationsartefakte sind häufig erreichbar, während Execution observed-Ausweise die methodisch engste Stufe darstellen.
- Chat-Interaktion ist kein Ersatz für Ergebnisabgleich: Interaktionsintensität und Prompt-Formen sind ohne Artefakt- und Statusbezug nicht belastbar als Effizienz- oder Qualitätssignale.
- Grenzen vereinfachter Produktivitätsnarrative: Produktivität ist in der untersuchten Umgebung nicht auf Generierungsgeschwindigkeit zu reduzieren, sondern entlang von Integrationsaufwand und Abgleichsarbeit einzuordnen.
- Relevanz von Triangulation statt Einzelmetriken: Belastbare Einordnung entsteht durch das Zusammenspiel von Pipeline-Status, Code- und Ausführungsartefakten sowie Chat-Mustern.

8.4. Beitrag der Arbeit

Wissenschaftlicher Beitrag. Die Arbeit dokumentiert eine strukturierte Einordnung der Auswertung eines KI-Assistenzsystems im Vibe-Coding-Kontext, indem Kompetenzgruppen als Analyseperspektiven mit Artefaktspuren und Prozesssignalen zusammengeführt werden.

Konzeptueller Beitrag. Die Arbeit operationalisiert Aufgabenerfolg über die Pipeline-Logik (Attempted, Implementedstatic, Execution observed; sowie Implemented* als abgeleitete Evidenzstufe) und verknüpft diese systematisch mit der Triangulation mehrerer Artefaktquellen. Damit liegt ein Auswertungsschema vor, das die Trennung zwischen intendierter Bearbeitung, sichtbarer Implementierung und Execution observed-Ausweisen explizit und nachvollziehbar macht.

Konzeptioneller Beitrag. Die Arbeit betont Kompetenz als Perspektive auf Nutzungstypen und verschiebt den Fokus von einer rein toolzentrierten Betrachtung hin zu der Frage, wie KI-Assistenzsysteme in konkreten Entwicklungspraktiken genutzt und eingeordnet werden.

Empirischer Beitrag. Die Sandbox-Studie in einer Brownfield-Umgebung umfasst eine Artefaktbasis aus Code-, Ausführungs- und Chat-Spuren, anhand derer die Pipeline-Status operationalisiert und gruppenbezogen eingeordnet werden können (Kapitel 6/7).

Kontextueller Bezug. Im Manuskript wird ein Rahmen zur Einführung und Auswertung von KI-Coding-Tools beschrieben, der Qualifizierung, Review-Abläufe und automatisierte Ausführung als Bestandteile einer kontrollierten Nutzung sichtbar macht.

8.5. Einordnung im BuyIn-Kontext

Die folgende Einordnung wird ausschließlich aus den in Kapitel 6 berichteten Mustern und der Einordnung in Kapitel 7 abgeleitet. Zentral ist dabei die Trennung zwischen Intention (Attempted), sichtbarer Implementierung (Implementedstatic) und Execution observed-Ausweisen (Execution observed). Insbesondere der wiederkehrende Engpass in Execution observed-Ausweisen bei Integrationsaufgaben (z.B. T4/T5) macht die Differenz zwischen Codegenerierung sowie Integration und Abgleich im Setting sichtbar.

8.6. Abschließendes Fazit

In dieser Arbeit wird die Auswertung KI-gestützter Softwareentwicklung im Vibe-Coding-Kontext über die Trennung zwischen Intention, Implementationsartefakt und Execution observed-Ausweisen strukturiert. Die Kombination aus Pipeline-Logik, Artefaktanalyse und Chat-Auswertung dient als Grundlage für eine konsistente, methodisch begründete Zusammenführung der Befunde (Kapitel 6) und ihrer Einordnung (Kapitel 7).

Im untersuchten Brownfield-Setting ist der Nutzen nicht als automatische Eigenschaft eines Tools zu verstehen, sondern als Zusammenspiel von Integrationsaufwand, Abgleichsarbeit und Nutzungstypen im Kompetenzkontext.

Einordnung (Group comparability & Confounder). Die Schlussfolgerungen sind an die in Kapitel 6 berichteten Proxys (Pipeline-Status, Code-/CI-Surrogate, Chat-Muster) gebunden; mögliche Konfundierungen und die konservative Lesart werden in Abschnitt 7.2 (Kapitel 7) begründet. Entsprechend begründen die Befunde keine kausalen Effekte und keine generalisierbaren Rangfolgen zwischen Kompetenzgruppen. Die Aussagen beziehen sich auf Branches als Analyseeinheiten und auf aggregierte Muster, nicht auf eine Leistungsbewertung einzelner Personen.

Zusammenfassung.

In dieser Arbeit werden singuläre Produktivitäts- oder Proxy-Metriken als nur begrenzt belastbar eingeordnet, weil Integrations- und Abgleichsarbeit zentral bleibt (Kapitel 6/7). Die Auswertung wird über die konsequente Trennung und Triangulation von Intention (Attempted), Implementationsartefakt (Implementedstatic) und Execution observed-Ausweisen (Execution observed) strukturiert. Kompetenz wird dabei durchgehend als Analyseperspektive genutzt, um unterschiedliche Nutzungs- und Abgleichsmodi einzuordnen, ohne Leistungsrangfolgen abzuleiten. Die Kernaussage ist: Belastbare Schlussfolgerungen entstehen erst aus der gemeinsamen Betrachtung dieser drei Ebenen. Übergreifend zeigen die Befunde, dass Attempted, Implementedstatic und Execution observed als unterscheidbare Statuskategorien nicht notwendigerweise zusammenfallen.

Literatur

- Anderson, John R. (1996). *The Architecture of Cognition*. Mahwah, NJ: Psychology Press.
- Barke, Shraddha, Michael B. James und Nadia Polikarpova (2023). „Grounded Copilot: How Programmers Interact with Code-Generating Models“. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1, S. 1–27. DOI: 10.1145/3586030.
- Bavarian, Mohammad u. a. (2022). „Efficient Training of Language Models to Fill in the Middle“. In: *arXiv preprint arXiv:2207.14255*. DOI: 10.48550/arXiv.2207.14255. arXiv: 2207.14255. URL: <https://arxiv.org/abs/2207.14255> (besucht am 02.01.2026).
- Brandtner, Matthias, Philipp Leitner und Cor-Paul Bezemer (2024). „Challenges of Brownfield Software Modernization in Practice: A Systematic Review“. Unpublished manuscript.
- Collins, Allan, John Seely Brown und Susan E. Newman (1989). „Cognitive Apprenticeship: Teaching the Crafts of Reading, Writing, and Mathematics“. In: *Knowing, Learning, and Instruction: Essays in Honor of Robert Glaser*. Hrsg. von Lauren B. Resnick. Lawrence Erlbaum, S. 453–494.
- Cook, Thomas D. und Donald T. Campbell (1979). *Quasi-Experimentation: Design & Analysis Issues for Field Settings*. Boston: Houghton Mifflin.
- Dreyfus, Hubert L. und Stuart E. Dreyfus (1986). *Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. New York: Free Press.
- Forsgren, Nicole, Margaret-Anne Storey und Thomas Zimmermann (2021). „SPACE: A Framework for Understanding Productivity in Software Engineering“. In: *Communications of the ACM* 64.6, S. 33–36. DOI: 10.1145/3454122.
- Green, Thomas R. G. (1989). „Cognitive Dimensions of Notations“. In: *People and Computers V*. Hrsg. von Alistair Sutcliffe und Linda Macaulay. Cambridge: Cambridge University Press, S. 443–460.
- Horvitz, Eric (1999). „Principles of Mixed-Initiative User Interfaces“. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, S. 159–166. DOI: 10.1145/302979.303029.
- ISO/IEC (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Standard. URL: <https://www.iso.org/standard/35733.html> (besucht am 02.01.2026).
- Kahneman, Daniel (2011). *Thinking, Fast and Slow*. New York: Farrar, Straus und Giroux.
- Lave, Jean und Etienne Wenger (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge: Cambridge University Press.

- Lee, John D. und Katrina A. See (2004). „Trust in Automation: Designing for Appropriate Reliance“. In: *Human Factors* 46.1, S. 50–80. DOI: 10.1518/hfes.46.1.50.
- Mozannar, Hussein, Gagan Bansal, Adam Fourney und Eric Horvitz (2022). „Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming“. In: *arXiv preprint arXiv:2210.14306*. DOI: 10.48550/arXiv.2210.14306. arXiv: 2210.14306. URL: <https://arxiv.org/abs/2210.14306> (besucht am 02.01.2026).
- Nadi, Sarah u. a. (2022). „Human-AI Collaboration in Software Engineering: Challenges and Opportunities“. In: *arXiv preprint arXiv:2206.01081*. DOI: 10.48550/arXiv.2206.01081. arXiv: 2206.01081. URL: <https://arxiv.org/abs/2206.01081> (besucht am 02.01.2026).
- Parasuraman, Raja und Dietrich H. Manzey (2010). „Complacency and Bias in Human Interaction with Automation: An Attentional Integration“. In: *Human Factors* 52.3, S. 381–410. DOI: 10.1177/0018720810376055.
- Pearce, Henry u. a. (2022). „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions“. In: *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, S. 754–768. DOI: 10.1109/SP46214.2022.9833571.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon und Mert Demirel (2023). „The impact of AI on developer productivity: Evidence from GitHub Copilot“. In: *arXiv preprint arXiv:2302.06590*. DOI: 10.48550/arXiv.2302.06590. arXiv: 2302.06590. URL: <https://arxiv.org/abs/2302.06590> (besucht am 02.01.2026).
- Saretsky, Gary (1972). „The OEO P.C. Experiment and the John Henry Effect“. In: *Phi Delta Kappan* 53.9, S. 579–581.
- Sweller, John (1988). „Cognitive Load During Problem Solving: Effects on Learning“. In: *Cognitive Science* 12.2, S. 257–285. DOI: 10.1207/s15516709cog1202_4.
- Vaithilingam, Priyan u. a. (2022). „Expectations vs. Reality: Evaluating Code Completion Tools in Realistic Settings“. In: *CHI Conference on Human Factors in Computing Systems*. ACM, S. 1–17. DOI: 10.1145/3491102.3502031.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser und Illia Polosukhin (2017). „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems*. Bd. 30, S. 5998–6008.
- Witte, Thomas u. a. (2023). „Context Windows and the Limits of AI-Assisted Programming“. In: *arXiv preprint arXiv:2308.10253*. DOI: 10.48550/arXiv.2308.10253. arXiv: 2308.10253. URL: <https://arxiv.org/abs/2308.10253> (besucht am 02.01.2026).
- Wohlin, Claes, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell und Anders Wesslén (2012). *Experimentation in Software Engineering*. Springer. DOI: 10.1007/978-3-642-29044-2.

A. Anhang

A.1. Dateiliste der in der Arbeit behandelten Artefakte

Stand: 02.01.2026

Diese Übersicht dokumentiert die in der Bachelorarbeit referenzierten Dateien, die auf der Abgabe-CD enthalten sind. Die aufgeführten Artefakte dienen der Ergebnisdarstellung, der ergänzenden Analyse sowie der Nachvollziehbarkeit der beschriebenen Auswertungen.

A.2. A) Zentrale Auswertungsartefakte

Die folgenden Dateien bilden die Grundlage für die in Kapitel 5 dargestellten Ergebnisse und aggregierten Auswertungen.

- `evaluation/result.json`
- `evaluation/result.md`
- `evaluation/task_pipeline_v3.json`
- `evaluation/task_pipeline_v3.csv`
- `evaluation/survey_mapping.json`

A.3. B) Weitere Analysen und Metriken

Diese Artefakte ergänzen die zentrale Auswertung um vertiefende Analysen und zusätzliche Metriken.

- `evaluation/deep_code_analysis.json`
- `evaluation/code_quality_metrics.json`
- `evaluation/data_safe/lint_results.json`
- `evaluation/data_safe/test_results.json`

A.4. C) Auswerteskripte (Reproduktionspfad)

Die folgenden Skripte implementieren die in der Arbeit beschriebenen Auswertungs- und Aggregationsschritte.

- `evaluation/run.js`
- `evaluation/run_pipeline_v3.js`
- `evaluation/map_survey.js`